

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРИКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2026 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного
забезпечення інформаційних систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Програмне забезпечення онлайн платформи мініігор з
підтримкою мультиплеєрної взаємодії та системи віртуальних
активів. Програмне забезпечення динамічної
інтеграції сторонніх ігрових модулів у вигляді плагінів

Виконали студент IV курсу, групи ІІ-21
(шифр групи)
Павлусенко Данило Олегович
(прізвище, ім'я, по батькові) _____ (підпис)

Керівник професор, д.т.н., проф., засл. діяч, Павлов О.А.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Консультант ст. викл., д-р філософії, Головченко М.М.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Рецензент доцент кафедри ІСТ, к.т.н., доц., Солдатова М.О.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Засвідчую, що у цьому дипломному проєкті
немає запозичень з праць інших авторів без
відповідних посилань.

Студент _____
(підпис)

Київ – 2026

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії
Рівень вищої освіти – перший (бакалаврський)
Спеціальність – 121 Інженерія програмного забезпечення
Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРИКОВ
(ім'я прізвище)

“ ____ ” _____ 2026 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Павлусенку Данилу Олеговичу
(прізвище, ім'я, по батькові)

1. Тема проєкту Програмне забезпечення онлайн платформи мініігор з підтримкою мультиплеєрної взаємодії та системи віртуальних активів. Програмне забезпечення динамічної інтеграції сторонніх ігрових модулів у вигляді плагінів
- керівник проєкту Павлов Олександр Анатолійович, д.т.н., засл. діяч, професор
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «3» червня 2026 р. №2055-с

2. Термін подання студентом проєкту «13» червня 2026 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Передпроєктне обстеження предметної області: постановка завдання дипломного проєктування з розмежуванням спільної та індивідуальної частин; аналіз і моделювання ключових бізнес-процесів реалізованої частини засобами нотації BPMN.

2) Розроблення вимог до програмного забезпечення: формування варіантів використання та опис основних сценаріїв взаємодії користувачів із системою; розроблення функціональних і нефункціональних вимог із визначенням їх пріоритетів та ризиків реалізації; постановка завдання на розробку програмного забезпечення.

3) Конструювання та розроблення програмного забезпечення: проєктування архітектури за моделлю C4; обґрунтування архітектурних рішень, технологій та засобів розробки; конструювання сервісів PluginSystem і Payment; опис структури бази даних.

4) Аналіз якості та тестування програмного забезпечення: опис процесів автоматизованого модульного та мануального тестування; контрольний приклад динамічного підключення сторонньої мінігри; оцінювання відповідності реалізації сформованим функціональним і нефункціональним вимогам.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використання

2) Схема структурна компонентів програмного забезпечення

3) Схема бізнес-процесу проходження мінігри

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «15» березня 2026 року

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	15.03.2026	
2	Аналіз існуючих методів розв'язання задачі	20.03.2026	
3	Постановка та формалізація задачі	04.04.2026	
4	Розробка інформаційного забезпечення	18.04.2026	
5	Алгоритмізація задачі	21.04.2026	
6	Обґрунтування вибору використаних технічних засобів	26.04.2026	
7	Розробка програмного забезпечення	13.05.2026	
8	Налагодження програми	24.05.2026	
9	Виконання графічних документів	27.05.2026	
10	Оформлення пояснювальної записки	29.05.2026	
11	Подання ДП на попередній захист	02.06.2026	
12	Подання ДП рецензенту	10.06.2026	
13	Подання ДП на основний захист	13.06.2026	

Студент

(підпис)

Данило ПАВЛУСЕНКО

(ініціали, прізвище)

Керівник

(підпис)

Олександр ПАВЛОВ

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проекту складається з чотирьох розділів, містить 31 таблицю, 10 рисунків та 30 джерел – загалом 63 сторінки.

Дипломний проект присвячений розробці двох серверних сервісів онлайн-платформи мініігор Quizanchos: підсистеми динамічної інтеграції сторонніх ігрових модулів у вигляді плагінів (PluginSystem) та сервісу поповнення внутрішнього балансу користувача криптовалютою USDT (Payment). Особливістю розробки є плагінна архітектура, що дає змогу підключати сторонні мінігри як окремі DLL-модулі без зміни та перекомпіляції ядра платформи.

Метою розробки є забезпечення розширюваності платформи без зміни її ядра завдяки динамічному підвантаженню сторонніх мініігор через визначені програмні інтерфейси, а також надання користувачам надійного та безпечного способу поповнення внутрішнього балансу криптовалютою з автоматичним виявленням і зарахуванням платежів.

У першому розділі виконано аналіз предметної області, порівняння аналогів та моделювання бізнес-процесів.

У другому розділі сформульовано 17 варіантів використання, 20 функціональних вимог, а також нефункціональні вимоги до реалізованої частини.

У третьому розділі розроблено архітектуру за моделлю C4, обґрунтовано вибір технологій (ASP.NET Core, EF Core, PostgreSQL), спроектовано алгоритми динамічного завантаження DLL-плагінів та поповнення балансу через криптовалютну біржу, а також структуру бази даних.

У четвертому розділі розроблено 31 модульний тест на базі фреймворка xUnit.net, виконано серію мануальних тестів та контрольний приклад динамічного підключення мінігри.

КЛЮЧОВІ СЛОВА: ОНЛАЙН-ПЛАТФОРМА, МІНІГРИ, ПЛАГІННА
АРХІТЕКТУРА, ДИНАМІЧНЕ ЗАВАНТАЖЕННЯ ЗБІРОК, ASP.NET CORE,
.NET, КРИПТОВАЛЮТА, USDT, BINANCE API, POSTGRESQL.

ABSTRACT

The explanatory note of the diploma project consists of four sections, contains 31 tables, 10 figures and 30 sources – 63 pages in total.

The diploma project is devoted to the development of two server-side services of the Quizanchos online minigames platform: a subsystem for dynamic integration of third-party game modules as plugins (PluginSystem) and a service for topping up the user's internal balance with the USDT cryptocurrency (Payment). A distinctive feature of the development is a plugin architecture that enables the integration of third-party minigames as separate DLL modules without modifying or recompiling the platform core.

The purpose of the project is to ensure the extensibility of the platform without modifying its core through dynamic loading of third-party minigames via defined programming interfaces, as well as to provide users with a reliable and secure means of topping up their internal balance with cryptocurrency, featuring automatic detection and crediting of payments.

The first section provides analysis of the subject area, comparison of analogous platforms and modelling of business processes.

The second section formulates 17 use cases, 20 functional requirements and a set of non-functional requirements for the implemented part.

The third section develops the architecture using the C4 model, justifies the choice of technologies (ASP.NET Core, EF Core, PostgreSQL), designs the algorithms for dynamic loading of DLL plugins and for balance top-up via a cryptocurrency exchange, and the database structure.

The fourth section develops 31 unit tests based on xUnit.net, performs a series of manual tests and executes the control example of dynamically connecting a minigame.

KEYWORDS: ONLINE PLATFORM, MINIGAMES, PLUGIN ARCHITECTURE, DYNAMIC ASSEMBLY LOADING, ASP.NET CORE, .NET, CRYPTOCURRENCY, USDT, BINANCE API, POSTGRESQL

[illegible]

Пояснювальна записка до дипломного проєкту

на тему: Програмне забезпечення онлайн платформи мініігор з підтримкою
мультиплеєрної взаємодії та системи віртуальних активів. Програмне
забезпечення динамічної інтеграції сторонніх ігрових модулів у
вигляді плагінів

КПІ.ПІ-2116.045440.02.81

ЗМІСТ

ВСТУП	4
1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	6
1.1 Постановка завдання дипломного проєктування	6
1.2 Аналіз та моделювання бізнес-процесів.....	7
Висновки до розділу	10
2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	11
2.1 Варіанти використання програмного забезпечення	11
2.2 Розроблення функціональних вимог	21
2.3 Розроблення нефункціональних вимог.....	26
2.4 Постановка завдання на розробку програмного забезпечення	28
Висновки до розділу	29
3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	32
3.1 Архітектура програмного забезпечення.....	32
3.2 Архітектурні рішення та обґрунтування вибору засобів розробки	33
3.3 Конструювання програмного забезпечення.....	35
3.4 Опис структури бази даних.....	38
Висновки до розділу	39
4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	41
4.1 Опис процесів тестування.....	41
4.2 Опис контрольного прикладу	48
Висновки до розділу	53
ВИСНОВКИ.....	56
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	58
ДОДАТОК А ЗВІТ ПОДІБНОСТІ.....	61

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated development environment
API	– Application programming interface
SDK	– Software development kit
IT	– Інформаційні технології
ER	– Entity-Relation diagram
ОС	– Операційна система
БД	– База даних
DQL	– Data query language
DML	– Data manipulation language
SQL	– Structured query language
DDL	– Data definition language
ORM	– Object-Relational Mapping
MVC	– Model-View-Controller
DI	– Dependency injection
UC	– Use Case
HTTP	– Протокол для обміну даними в Інтернеті
HTTPS	– HTTP з шифруванням для безпечного обміну даними
URL	– Адреса для доступу до ресурсів в Інтернеті

ВСТУП

Онлайн-платформи мініігор є важливою складовою сучасного цифрового середовища, в якому попит на інтерактивні сервіси та ігрову взаємодію постійно зростає [1]. Конкурентоспроможність таких платформ значною мірою визначається широтою й різноманітністю каталогу доступних ігор, а отже – здатністю системи швидко та безпечно поповнюватися новими ігровими модулями, у тому числі створеними сторонніми розробниками. Це висуває на перший план питання архітектурної гнучкості та розширюваності платформи.

Традиційний підхід, за якого додавання кожної нової мінігри потребує зміни та повторної компіляції коду ядра платформи, погано масштабується: він підвищує бар'єр входу для зовнішніх розробників, уповільнює оновлення каталогу та збільшує ризик дестабілізації системи в цілому. Тому актуальним є застосування архітектурних рішень, що мінімізують жорсткі залежності між компонентами й дозволяють розширювати функціональність без втручання в існуючий код.

Ефективним способом розв'язання цієї задачі є плагінна архітектура, що ґрунтується на принципі слабкої зв'язності (loose coupling) – взаємодії частин системи через визначені інтерфейси без прямої прив'язки до їхньої внутрішньої реалізації [2]. За такого підходу кожна мінігра постачається як окремий модуль, що динамічно підключається до платформи у вигляді скомпільованої збірки й самостійно містить ігрову логіку та клієнтські ресурси, тоді як платформа надає готову інфраструктуру для його роботи. Динамічне завантаження та ізоляція стороннього коду під час виконання [3] дозволяють розширювати каталог мініігор без зміни ядра, підвищуючи надійність, спрощуючи супровід та забезпечуючи стабільну роботу при зростанні навантаження.

Не менш важливою складовою сучасних ігрових платформ є система внутрішніх віртуальних активів та внутрішньоігрової валюти, що формує економічну модель середовища й підвищує залученість користувачів [4].

Поповнення внутрішнього балансу традиційно реалізується через сторонні платіжні процесори, які стягують комісію, висувають вимоги до верифікації користувача та створюють залежність від зовнішнього сервісу. Альтернативою є прямий прийом криптовалюти, зокрема стейблкоїна USDT, що знижує витрати, спрощує аудит операцій завдяки прозорості блокчейну та усуває залежність від процесингової платформи.

З огляду на викладене, актуальною є розробка програмного забезпечення, що поєднує гнучке розширення каталогу мініігор із надійним механізмом монетизації. У межах індивідуальної частини дипломного проєкту розв'язанню цих задач присвячено два серверні сервіси онлайн-платформи мініігор Quizanchos – підсистему динамічної інтеграції сторонніх ігрових модулів у вигляді плагінів (PluginSystem), що забезпечує підключення нових мініігор без зміни коду ядра, та сервіс поповнення внутрішнього балансу користувача криптовалютою USDT (Payment). Проєктування й реалізація цих сервісів засобами платформи .NET та ASP.NET Core є предметом подальшого розгляду в наступних розділах записки.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Постановка завдання дипломного проєктування

Комплексний дипломний проєкт передбачає вивчення предметної області, проєктування та створення програмного забезпечення з подальшим оцінюванням його ефективності, безпеки й якості. Роботу над проєктом структуровано у два блоки: спільний, який стосується загальних питань побудови системи, та індивідуальний, зосереджений на проєктуванні, реалізації й тестуванні окремого функціонального модуля програмного продукту.

У межах спільної частини дипломного проєкту було виконано:

- проведення аналізу предметної області, визначення її ключових бізнес-процесів та формування загальної постановки завдання розробки;
- аналіз існуючих програмних продуктів, алгоритмічних і технічних рішень, що реалізують подібні функції;
- формування системних вимог до програмного забезпечення;
- проведення економічного аналізу розроблюваного програмного продукту;
- забезпечення інформаційної безпеки та захисту даних;
- розгортання програмного забезпечення та організація його супроводу.

В індивідуальній частині дипломного проєкту поставлено такі задачі:

- моделювання та оптимізація бізнес-процесів предметної області у частині, що стосується реалізованого функціоналу;
- формування функціональних та нефункціональних вимог до реалізованого функціоналу програмного забезпечення;
- постановка технічного завдання на розробку відповідної частини програмного забезпечення;
- проєктування архітектури реалізованої частини програмного забезпечення;

- обґрунтування вибору архітектурних рішень, технологій та засобів розробки;
- реалізація програмного забезпечення відповідно до сформованих вимог;
- проведення тестування та оцінювання якості реалізованого програмного продукту;
- розроблення супровідної та технічної документації індивідуальної частини.

1.2 Аналіз та моделювання бізнес-процесів

Для опису бізнес-процесів використано BPMN-моделі [5] (рисунки 1.1-1.2).

Опис послідовності розгортання нової мінігри на платформі (діаграма наведена у графічних матеріалах “Схема бізнес-процесу розгортання нової мінігри розробником на платформі Quizanchos”):

- розробник завершує підготовку нової мінігри та формує її готовий пакет із правилами, описом і інтерфейсом;
- сформований пакет мінігри передається на платформу разом із екраном та оформленням, які бачитимуть гравці;
- після передачі всіх артефактів розробник публікує оновлення платформи, ініціюючи процес розгортання нової мінігри;
- платформа готує сховище даних та завантажує файли, необхідні для роботи нової мінігри;
- система виконує пошук нової мінігри серед встановлених модулів і зчитує її опис: назву, тип, ознаку преміум-доступу та правила;
- на основі заявленого переліку послуг платформа підключає внутрішні служби, які потрібні мінігрі для коректної роботи;
- платформа отримує опис можливих ходів гравців і налаштовує власні механізми розпізнавання цих ходів у новій мінігрі;

- після успішного підключення мінігра додається до каталогу доступних ігор, що робить її видимою для користувачів;
- процес завершується, коли платформа готова приймати гравців у нову мінігру, а сама мінігра з'являється у каталозі для користувачів.

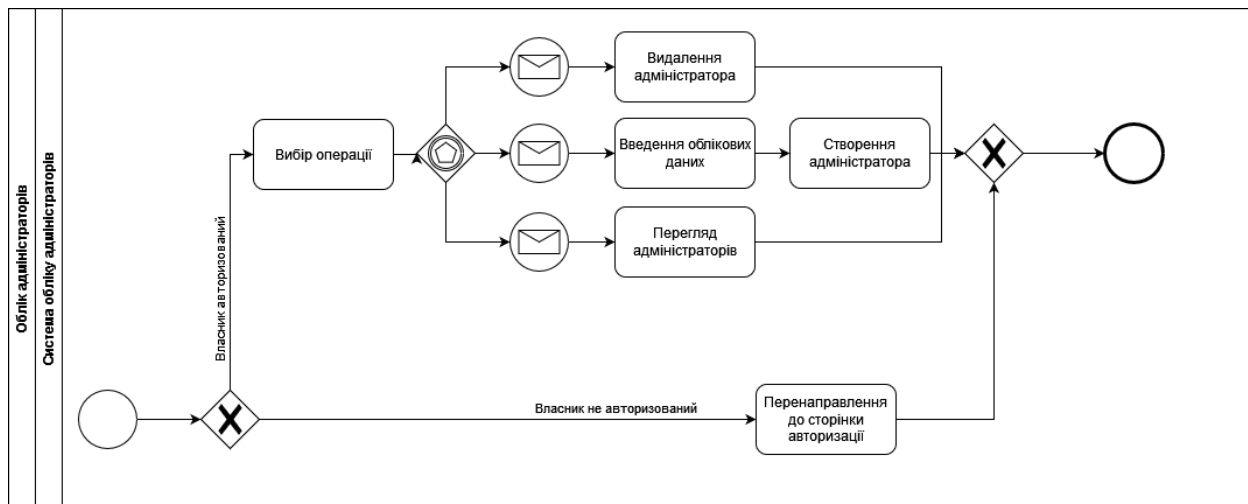


Рисунок 1.1 – Схема бізнес-процесу обліку адміністраторів

Опис послідовності обліку адміністраторів:

- власник здійснює авторизацію у системі;
- якщо власник не авторизований, застосунок перенаправляє його на сторінку авторизації;
- після успішної авторизації власник обирає необхідну операцію (наприклад, видалення адміністратора, введення облікових даних, перегляд існуючих адміністраторів або створення нового адміністратора).

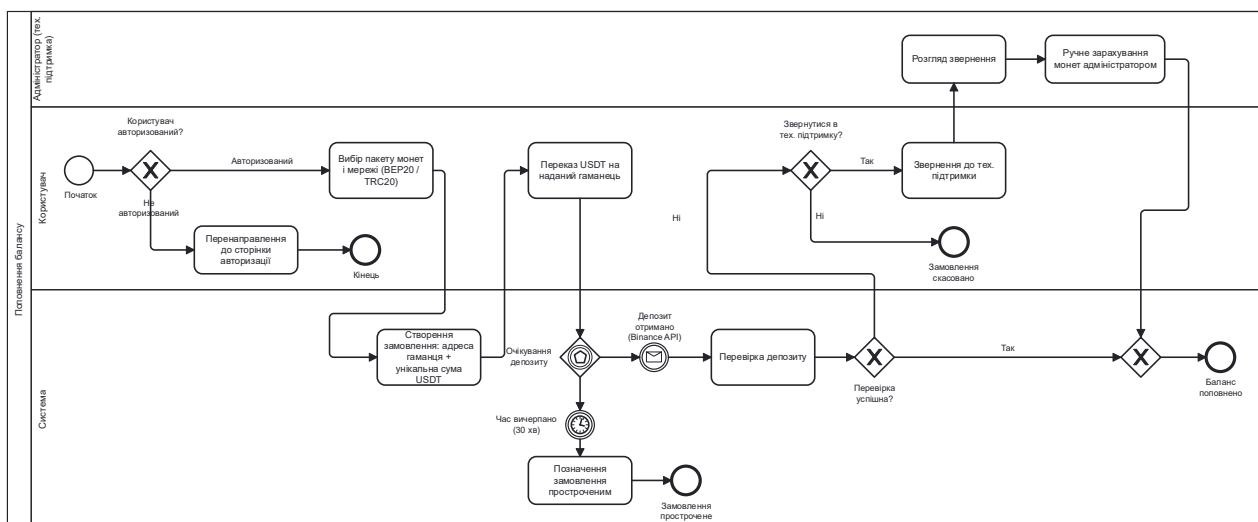


Рисунок 1.2 – Схема бізнес-процесу поповнення балансу користувачем

Опис послідовності поповнення балансу користувачем:

- користувач ініціює поповнення балансу, після чого система перевіряє, чи він авторизований; якщо ні – користувача перенаправляє на сторінку авторизації, і процес на цьому завершується;
- авторизований користувач обирає пакет монет і мережу переказу (BER20 або TRC20) [6], відповідно до власних потреб;
- на основі обраного пакету система створює замовлення, формуючи адресу гаманця та унікальну суму USDT [7], за якою платіж буде однозначно ідентифіковано;
- користувач виконує переказ USDT на надану адресу гаманця у вказаній мережі;
- після переказу система переходить у режим очікування депозиту, відстежуючи дві можливі події: надходження коштів або вичерпання відведеного часу;
- якщо протягом 30 хвилин кошти не надходять, спрацьовує подія тайм-ауту, система позначає замовлення простроченим, і процес завершується статусом «замовлення прострочене»;
- якщо ж від платіжної системи надходить повідомлення про депозит (через Binance API) [8], система виконує перевірку депозиту на відповідність очікуваній сумі та мережі [9];
- за результатом перевірки система визначає, чи вона успішна: у разі успіху баланс зараховується автоматично, потоки об'єднуються, і процес завершується статусом «баланс поповнено»;
- якщо перевірка неуспішна, користувачеві пропонується звернутися до технічної підтримки; за відмови процес завершується статусом «замовлення скасовано»;
- у разі звернення користувач створює запит до технічної підтримки, адміністратор розглядає звернення та виконує ручне зарахування монет, після чого потоки об'єднуються;

– процес завершується, коли баланс користувача успішно поповнено – автоматично після перевірки депозиту або вручну за результатом розгляду звернення адміністратором.

Висновки до розділу

У першому розділі виконано передпроектне обстеження предметної області онлайн-платформ мініігор, які поєднують багатокористувацьку ігрову взаємодію, елементи гейміфікації та систему віртуальних активів. Визначено, що ключовими чинниками якості таких платформ є гнучка інтеграція ігрових компонентів із мінімізацією жорстких залежностей між ними та масштабованість системи. На основі цього сформульовано постановку завдання комплексного дипломного проєкту з розмежуванням спільної та індивідуальної частин, у межах якого до індивідуальної частини віднесено моделювання бізнес-процесів, формування вимог, проєктування, реалізацію та тестування відповідного функціоналу.

Виконано аналіз та моделювання ключових бізнес-процесів реалізованої частини засобами нотації BPMN. Зокрема, формалізовано процес розгортання нової мінігри розробником на платформі – від підготовки пакета до появи мінігри в каталозі, процес обліку адміністраторів, а також процес поповнення внутрішнього балансу користувачем через криптовалютний переказ USDT з автоматичною перевіркою депозиту та альтернативним ручним зарахуванням за участі технічної підтримки.

Побудовані моделі бізнес-процесів унаочнюють логіку взаємодії учасників із системою та визначають межі автоматизації, що становить основу для формування варіантів використання, функціональних і нефункціональних вимог у наступному розділі.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Головною функцією програмного забезпечення індивідуальної частини є динамічне підключення сторонніх мініігор у вигляді плагінів та поповнення внутрішнього балансу користувача криптовалютою USDT, а також адміністрування й моніторинг відповідних сервісів платформи. Більше функцій можна побачити на діаграмі варіантів використання у графічних матеріалах “Схема структурна варіантів використання”.

В таблицях 2.1-2.17 наведено опис варіантів використання програмного забезпечення.

Таблиця 2.1 – Варіант використання UC-01

Use case name	Завантаження пакету мінігри
Use case ID	UC-01
Goals	Передати на платформу готовий пакет власної мінігри (DLL-збірка, маніфест plugin.json та статичні ресурси) для подальшої реєстрації
Actors	Розробник
Trigger	Розробник завершив підготовку мінігри та має готовий пакет для розміщення на платформі
Pre-conditions	Розробник має доступ до каталогу плагінів сервера (Plugins:Root); мінігра зібрана на основі SDK-пакета Quizanchos.Core
Flow of Events	Розробник формує піддиректорію плагіна з вхідною збіркою, файлом-маніфестом plugin.json (ім'я вхідного асемблі, шлях до wwwroot) та, за потреби, піддиректорією wwwroot із клієнтськими ресурсами. Сформований пакет розміщується у каталозі плагінів сервера. На наступному циклі завантаження платформа виявляє нову піддиректорію та готує її до реєстрації. Варіант використання включає (include) реєстрацію мінігри в каталозі.

Продовження таблиці 2.1

Extension	Якщо файл <code>plugin.json</code> відсутній або містить помилки, платформа використовує значення за замовчуванням (входний асемблі за іменем папки, <code>wwwroot</code> за замовчуванням); помилка пакування одного плагіна не впливає на завантаження інших.
Post-Condition	Пакет мінігри розміщено у каталозі плагінів і готовий до завантаження та реєстрації платформою.

Таблиця 2.2 – Варіант використання UC-02

Use case name	Реєстрація мінігри в каталозі
Use case ID	UC-02
Goals	Зареєструвати завантажену мінігру у внутрішніх реєстрах платформи, щоб ядро могло маршрутизувати до неї ігрові запити
Actors	Розробник
Trigger	Пакет мінігри розміщено у каталозі плагінів і платформа виконує його завантаження
Pre-conditions	Пакет мінігри завантажено (UC-01); збірка реалізує контракти SDK-пакета <code>Quizanchos.Core</code> (UC-06)
Flow of Events	Платформа обгортає збірку у власний ізольований контекст завантаження (<code>PluginLoadContext</code>) і засобами рефлексії знаходить класи, що реалізують <code>IMinigameDescriptor</code> та <code>IMinigameFrontendDescriptor</code> . Знайдені дескриптори інстанціюються та реєструються у глобальних реєстрах <code>IMinigameRegistry</code> і <code>IMinigameFrontendRegistry</code> , після чого зіставляються з рядком каталогу типів мініігор (<code>game_type</code>) за ключем <code>gameKey</code> . Варіант використання включає (<code>include</code>) завантаження пакету мінігри та підтримку NuGet-пакета інтерфейсу мінігри.
Extension	Якщо <code>MinigameTypeId</code> дескриптора менший за зарезервований поріг (1000), плагін відхиляється повністю, щоб унеможливити перевизначення вбудованих ідентифікаторів мініігор. Помилки завантаження окремих типів логуються, а решта типів продовжує реєструватися.
Post-Condition	Мінігра зареєстрована у реєстрах платформи, ядро здатне маршрутизувати до неї запити, проте для гравців вона ще не опублікована.

Таблиця 2.3 – Варіант використання UC-03

Use case name	Публікація мінігри для гравців
Use case ID	UC-03
Goals	Зробити зареєстровану мінігру видимою та доступною гравцям у публічному каталозі
Actors	Розробник
Trigger	Розробник бажає відкрити гравцям доступ до підготовленої мінігри
Pre-conditions	Мінігра зареєстрована у реєстрах платформи (UC-02)
Flow of Events	Розробник ініціює публікацію мінігри. Платформа монтує статичні ресурси плагіна (wwwroot) у публічний шлях /minigames/{gamekey}/, після чого мінігра з'являється у каталозі доступних ігор. Для застосування змін виконується перезавантаження web API серверу, під час якого PluginLoader повторно підвантажує плагіни. Варіант використання включає (include) реєстрацію мінігри в каталозі та перезавантаження web API серверу.
Extension	Якщо під час підвантаження мінігра не проходить валідацію, вона не публікується, а подію зафіксовано у журналі.
Post-Condition	Мінігра опублікована та відображається у публічному каталозі; гравці можуть її запускати.

Таблиця 2.4 – Варіант використання UC-04

Use case name	Редагування параметрів мінігри
Use case ID	UC-04
Goals	Змінити налаштування та метадані власної мінігри (опис, ознака преміум-доступу, шлях до ресурсів, локалізовані назви)
Actors	Розробник
Trigger	Розробник бажає скоригувати параметри раніше розміщеної мінігри
Pre-conditions	Розробник має доступ до пакета мінігри у каталозі плагінів

Продовження таблиці 2.4

Flow of Events	Розробник редагує маніфест plugin.json та супутні налаштування пакета (назва, опис, ознака преміум-доступу, локалізовані варіанти, шлях до wwwroot). Оновлений пакет розміщується у каталозі плагінів. Зміни застосовуються після наступного підвантаження плагінів платформою.
Extension	У разі некоректного маніфесту платформа повертається до значень за замовчуванням, а помилку зафіксовано у журналі.
Post-Condition	Параметри мінігри оновлено; після підвантаження платформа використовує нові значення.

Таблиця 2.5 – Варіант використання UC-05

Use case name	Перезавантаження web API серверу
Use case ID	UC-05
Goals	Перезапустити серверний застосунок, щоб платформа повторно підвантажила плагіни мініігор і застосувала зміни
Actors	Розробник
Trigger	Потрібно застосувати додавання, оновлення або вилучення плагінів мініігор
Pre-conditions	Розробник має право ініціювати перезапуск web API серверу
Flow of Events	Розробник ініціює перезавантаження серверного застосунку ASP.NET Core. На старті метод PluginLoader.LoadFromDirectory перебирає піддиректорії каталогу плагінів і заново підвантажує кожен плагін у власний ізольований контекст, реєструючи його дескриптори. Відсутність каталогу плагінів не блокує старт застосунку.
Extension	Помилка завантаження окремого плагіна логується й не зриває старту інших плагінів та застосунку в цілому.
Post-Condition	Серверний застосунок перезапущено, актуальний набір плагінів підвантажено та зареєстровано.

Таблиця 2.6 – Варіант використання UC-06

Use case name	Підтримка NuGet пакету інтерфейсу мінігри
Use case ID	UC-06

Продовження таблиці 2.6

Goals	Отримати та використати переносний SDK-пакет з контрактами інтеграції, на основі якого будується сумісна з платформою мінігра
Actors	Розробник
Trigger	Розробник розпочинає створення нової мінігри під платформу
Pre-conditions	Розробнику доступний NuGet-пакет Quizanchos.Core
Flow of Events	Розробник підключає до свого проєкту NuGet-пакет Quizanchos.Core, що містить базові інтерфейси контракту між мінігрою та ядром (IMinigameDescriptor, IGameLogic, IGameStatePersistence). Реалізуючи ці інтерфейси, розробник створює DLL-збірку мінігри без втручання у код ядра. Спільні з ядром типи завантажуються з контексту за замовчуванням, що гарантує сумісність версій між хостом і плагіном.
Extension	За невідповідності версії пакета контракту платформи дескриптори не розпізнаються під час реєстрації; розробнику потрібно оновити версію SDK-пакета.
Post-Condition	Мінігра реалізує контракт платформи й придатна до завантаження та реєстрації.

Таблиця 2.7 – Варіант використання UC-07

Use case name	Налаштування .env (гаманець)
Use case ID	UC-07
Goals	Налаштувати адреси криптовалютних гаманців для прийому поповнень балансу користувачів
Actors	Адміністратор
Trigger	Потрібно ввести або змінити адреси гаманців для прийому USDT
Pre-conditions	Адміністратор має доступ до конфігурації середовища (.env) серверного застосунку
Flow of Events	Адміністратор задає у файлі середовища (.env) адреси гаманців для мереж BEP20 та TRC20 (UsdtAddressBep20, UsdtAddressTrc20). Після перезапуску застосунку ці значення використовує сервіс Payment під час створення замовлень на поповнення балансу.

Продовження таблиці 2.7

Extension	За відсутності або некоректності адрес створення замовлень на поповнення стає неможливим, про що повідомляється у журналі.
Post-Condition	Адреси гаманців збережено у конфігурації; платформа може повертати їх користувачам у замовленнях на поповнення.

Таблиця 2.8 – Варіант використання UC-08

Use case name	Налаштування .env (API)
Use case ID	UC-08
Goals	Налаштувати параметри доступу до зовнішнього платіжного API (ключ і секрет Binance) для перевірки депозитів
Actors	Адміністратор
Trigger	Потрібно ввести або оновити облікові дані доступу до Binance API
Pre-conditions	Адміністратор має доступ до конфігурації середовища (.env)
Flow of Events	Адміністратор задає у файлі середовища (.env) параметри BinanceApiOptions (API-ключ та секрет). Секрет використовується для підпису HMAC-SHA256 запитів до Binance Sapi API під час перевірки депозитів і ніколи не зберігається у репозиторії коду.
Extension	За некоректних облікових даних фонові перевірки депозитів завершується помилкою HTTP, що логується, а наступний цикл повторює спробу.
Post-Condition	Параметри доступу до платіжного API збережено; фонові перевірки депозитів здатні звертатися до Binance API.

Таблиця 2.9 – Варіант використання UC-09

Use case name	Перегляд логів системи
Use case ID	UC-09
Goals	Переглянути журнали роботи платформи для діагностики та контролю стану системи
Actors	Адміністратор
Trigger	Адміністратор бажає перевірити журнали подій або діагностувати проблему
Pre-conditions	Адміністратор авторизований з правами доступу до журналів

Продовження таблиці 2.9

Flow of Events	Адміністратор відкриває розділ журналів. Система відображає зафіксовані події роботи платформи (завантаження плагінів, перевірку депозитів, помилки інтеграцій тощо) з можливістю фільтрації за рівнем та періодом.
Extension	Якщо за обраними критеріями записів немає, відображається інформаційне повідомлення.
Post-Condition	Адміністратор ознайомився з журналами; стан системи не змінився.

Таблиця 2.10 – Варіант використання UC-10

Use case name	Перегляд статистики платформи
Use case ID	UC-10
Goals	Ознайомитися з агрегованими показниками роботи платформи
Actors	Адміністратор
Trigger	Адміністратор бажає оцінити стан і активність платформи
Pre-conditions	Адміністратор авторизований з відповідними правами
Flow of Events	Адміністратор відкриває розділ статистики. Система відображає агреговані показники роботи платформи (кількість користувачів, активність мініігор, обсяг транзакцій тощо) з можливістю вибору періоду.
Extension	Якщо дані ще не накопичені, відображається інформаційне повідомлення.
Post-Condition	Адміністратор переглянув статистику; стан системи не змінився.

Таблиця 2.11 – Варіант використання UC-11

Use case name	Управління користувачами
Use case ID	UC-11
Goals	Керувати обліковими записами користувачів платформи
Actors	Адміністратор
Trigger	Адміністратор бажає переглянути або змінити стан облікового запису користувача
Pre-conditions	Адміністратор авторизований з відповідними правами

Продовження таблиці 2.11

Flow of Events	Адміністратор відкриває розділ управління користувачами, переглядає перелік облікових записів і виконує необхідні операції (блокування, розблокування, зміна ролей, перегляд деталей). Дії автоматично фіксуються у журналі.
Extension	Критичні дії потребують додаткового підтвердження; за недостатніх прав адміністратор отримує повідомлення про відмову.
Post-Condition	Зміни щодо облікового запису збережено; відповідний запис додано в журнал.

Таблиця 2.12 – Варіант використання UC-12

Use case name	Управління преміум-підписками
Use case ID	UC-12
Goals	Керувати преміум-підписками користувачів платформи
Actors	Адміністратор
Trigger	Адміністратор бажає переглянути або змінити статус преміум-підписки користувача
Pre-conditions	Адміністратор авторизований з відповідними правами
Flow of Events	Адміністратор відкриває розділ управління підписками, переглядає поточні преміум-підписки та за потреби активує, продовжує або скасовує підписку користувача. Зміни фіксуються у журналі.
Extension	За недостатніх прав або некоректних даних операція відхиляється з відповідним повідомленням.
Post-Condition	Статус преміум-підписки оновлено; відповідний запис додано в журнал.

Таблиця 2.13 – Варіант використання UC-13

Use case name	Адміністрування маркету
Use case ID	UC-13
Goals	Керувати асортиментом і параметрами внутрішньоігрового маркету
Actors	Адміністратор
Trigger	Адміністратор бажає змінити склад або налаштування предметів маркету

Продовження таблиці 2.13

Pre-conditions	Адміністратор авторизований з відповідними правами
Flow of Events	Адміністратор відкриває розділ адміністрування маркету, переглядає перелік предметів і виконує операції з ними (додавання, редагування, приховування, зміна ціни). Зміни фіксуються у журналі.
Extension	За некоректних параметрів предмета операція відхиляється з відповідним повідомленням.
Post-Condition	Зміни в маркеті збережено та відображені користувачам.

Таблиця 2.14 – Варіант використання UC-14

Use case name	Адміністрування рейтингу
Use case ID	UC-14
Goals	Керувати параметрами рейтингової системи та коригувати рейтингові дані
Actors	Адміністратор
Trigger	Адміністратор бажає налаштувати правила рейтингу або скоригувати рейтингові записи
Pre-conditions	Адміністратор авторизований з відповідними правами
Flow of Events	Адміністратор відкриває розділ адміністрування рейтингу, переглядає рейтингові таблиці та налаштовує правила нарахування або коригує окремі записи. Зміни фіксуються у журналі.
Extension	Критичні зміни потребують підтвердження; за недостатніх прав операція відхиляється.
Post-Condition	Параметри або дані рейтингу оновлено; відповідний запис додано в журнал.

Таблиця 2.15 – Варіант використання UC-15

Use case name	Перегляд історії транзакцій маркету
Use case ID	UC-15
Goals	Переглянути історію фінансових операцій і транзакцій маркету для контролю та аудиту
Actors	Адміністратор

Продовження таблиці 2.15

Trigger	Адміністратор бажає переглянути транзакції або опрацювати звернення користувача щодо поповнення
Pre-conditions	Адміністратор авторизований з відповідними правами
Flow of Events	Адміністратор відкриває розділ історії транзакцій. Система відображає перелік замовлень на поповнення та операцій маркету з їх статусами (Pending, Completed, Expired), сумами, мережами та ідентифікаторами. Адміністратор може фільтрувати транзакції за статусом, користувачем і періодом та переглядати деталі конкретної операції.
Extension	За наявності спірної або не зарахованої автоматично операції адміністратор може виконати ручне підтвердження транзакції (розширення extend, UC-16).
Post-Condition	Адміністратор ознайомився з історією транзакцій; за потреби ініційовано подальші дії.

Таблиця 2.16 – Варіант використання UC-16

Use case name	Ручне підтвердження транзакції
Use case ID	UC-16
Goals	Вручну зарахувати кошти за замовленням, яке не було підтверджено автоматично
Actors	Адміністратор
Trigger	Користувач звернувся до технічної підтримки щодо неавтоматично зарахованого поповнення, або адміністратор виявив таку транзакцію в історії
Pre-conditions	Існує замовлення на поповнення, депозит за яким не зараховано автоматично; адміністратор авторизований з відповідними правами
Flow of Events	Варіант використання розширює (extend) перегляд історії транзакцій маркету. Адміністратор відкриває деталі проблемного замовлення, перевіряє відповідність переказу (сума, мережа, TxId) і виконує ручне зарахування монет на баланс користувача. Операція виконується транзакційно: статус замовлення переводиться у завершений, а баланс користувача оновлюється. Дію зафіксовано у журналі.
Extension	Якщо переказ не підтверджено, адміністратор відхиляє звернення, а замовлення завершується статусом «скасовано».

Продовження таблиці 2.16

Post-Condition	Баланс користувача поповнено вручну, статус замовлення оновлено, відповідний запис додано в журнал.
----------------	---

Таблиця 2.17 – Варіант використання UC-17

Use case name	Поповнення балансу
Use case ID	UC-17
Goals	Поповнити внутрішній баланс монет шляхом криптовалютного переказу USDT
Actors	Авторизований користувач
Trigger	Користувач бажає поповнити баланс внутрішньоігрової валюти
Pre-conditions	Користувач авторизований; адміністратор налаштував адреси гаманців у конфігурації (UC-07)
Flow of Events	Користувач обирає пакет монет і мережу переказу (BEP20 або TRC20). Система створює замовлення, генеруючи унікальну суму USDT та повертаючи адресу гаманця відповідної мережі й час дії замовлення (30 хвилин). Користувач виконує переказ USDT на надану адресу. Фонове завдання періодично перевіряє депозити через Binance API; у разі збігу за парою (сума, мережа) замовлення переводиться у завершений стан і монети автоматично нараховуються на баланс. Варіант використання включає (include, cross-actor) налаштування адрес гаманців у .env, виконане адміністратором (UC-07).
Extension	Якщо протягом 30 хвилин кошти не надходять, замовлення позначається простроченим. Якщо автоматична перевірка неуспішна, користувачеві пропонується звернутися до технічної підтримки для ручного зарахування (UC-16); за відмови замовлення скасовується.
Post-Condition	Баланс користувача поповнено (автоматично або вручну), відповідний запис додано в історію транзакцій; або замовлення завершено статусом «прострочене»/«скасовано».

2.2 Розроблення функціональних вимог

Програмне забезпечення індивідуальної частини побудоване як набір функціонально відокремлених модулів, кожен з яких відповідає за окремий напрям роботи розглянутих сервісів [10]: інтеграцію мініігор від зовнішнього

розробника (сервіс PluginSystem), поповнення внутрішнього балансу та платіжні операції (сервіс Payment), а також адміністрування та моніторинг платформи. Таке розділення дозволяє виокремити вимоги до кожного компонента, оцінити їх важливість для роботи платформи в цілому та ризики, пов'язані з їх реалізацією.

Загальна модель вимог наведена в таблиці 2.18, а детальний опис кожної функціональної вимоги – у таблиці 2.19. Зв'язки між варіантами використання та функціональними вимогами відображені у матриці трасування на рисунку 2.1.

Пріоритет кожної вимоги визначався виходячи з її ролі у роботі розглянутих сервісів [11]. Високий пріоритет присвоєно вимогам, без яких неможливе виконання основних сценаріїв, а саме динамічного підключення сторонніх мініігор у вигляді DLL-модулів, їх реєстрації та публікації, конфігурування й роботи платіжного шлюзу та поповнення внутрішнього балансу через криптовалютний переказ. Середній пріоритет отримали вимоги, що розширюють можливості адміністрування та супроводу платформи (управління користувачами та підписками, адміністрування маркету й рейтингу, редагування параметрів мінігри, перегляд історії транзакцій), однак не блокують основну роботу системи в разі їх відсутності. Низький пріоритет присвоєно вимогам допоміжного характеру, зокрема перегляду логів і статистики платформи, які впливають переважно на зручність супроводу.

Оцінка ризиків відображає технічну складність реалізації кожної вимоги та її залежність від зовнішніх факторів. Високий ризик мають вимоги, пов'язані з динамічним підключенням стороннього коду у runtime та обробкою фінансово значущих операцій, зокрема завантаження й ізоляція DLL-збірок мініігор, реєстрація та публікація плагінів, перезавантаження web API серверу, інтеграція з платіжним шлюзом Binance, поповнення балансу та ручне підтвердження транзакцій. Середній ризик відповідає вимогам із помірною технічною складністю, де основна частина логіки реалізується засобами платформи, але присутні неочевидні граничні випадки (редагування

параметрів мінігри, управління підписками, адміністрування маркету й рейтингу, перегляд історії транзакцій). Низький ризик характерний для вимог, що зводяться до типових операцій з даними та перегляду інформації (перегляд логів і статистики) та не залежать від зовнішніх систем.

Таблиця 2.18 – Загальна модель вимог

№	Назва	ID Вимоги	Пріоритети	Ризики
1	Інтеграція мінігор від розробника	FR-1	Високий	Високий
1.1	Завантаження пакету мінігри	FR-2	Високий	Високий
1.2	Підтримка NuGet-пакета інтерфейсу мінігри	FR-3	Високий	Середній
1.3	Реєстрація мінігри в каталозі	FR-4	Високий	Високий
1.4	Публікація мінігри для гравців	FR-5	Високий	Високий
1.5	Редагування параметрів мінігри	FR-6	Середній	Середній
1.6	Перезавантаження web API серверу	FR-7	Високий	Високий
2	Поповнення балансу та платіжні операції	FR-8	Високий	Високий
2.1	Налаштування адрес гаманців у середовищі (.env)	FR-9	Високий	Високий
2.2	Налаштування доступу до платіжного API (.env)	FR-10	Високий	Високий
2.3	Поповнення балансу через переказ USDT	FR-11	Високий	Високий
2.4	Перегляд історії транзакцій	FR-12	Середній	Середній
2.5	Ручне підтвердження транзакції	FR-13	Високий	Високий
3	Адміністрування та моніторинг платформи	FR-14	Високий	Середній
3.1	Перегляд логів системи	FR-15	Низький	Низький
3.2	Перегляд статистики платформи	FR-16	Низький	Низький
3.3	Управління користувачами	FR-17	Середній	Високий
3.4	Управління преміум-підписками	FR-18	Середній	Середній
3.5	Адміністрування маркету	FR-19	Середній	Середній
3.6	Адміністрування рейтингу	FR-20	Середній	Середній

Таблиця 2.19 – Перелік функціональних вимог

Назва	Опис
FR-1	Інтеграція мініігор від розробника. Сукупність механізмів сервісу PluginSystem, що дозволяють зовнішньому розробнику підключити власну мінігру до платформи як окрему DLL-збірку без зміни коду ядра.
FR-2	Завантаження пакету мінігри. Розробник розміщує у каталозі плагінів пакет власної мінігри (DLL-збірка, маніфест plugin.json та статичні ресурси), який платформа підхоплює під час підвантаження.
FR-3	Підтримка NuGet-пакета інтерфейсу мінігри. Платформа надає переносний SDK-пакет Quizanchos.Core з контрактами IMinigameDescriptor, IGameLogic та IGameStatePersistence, на основі яких будується сумісна мінігра.
FR-4	Реєстрація мінігри в каталозі. Платформа засобами рефлексії виявляє у завантаженій збірці дескриптори мінігри та реєструє їх у реєстрах IMinigameRegistry і IMinigameFrontendRegistry із перевіркою діапазону MinigameTypeId ≥ 1000 .
FR-5	Публікація мінігри для гравців. Розробник відкриває гравцям доступ до зареєстрованої мінігри, після чого її статичні ресурси монтуються у публічний шлях і вона з'являється у каталозі доступних ігор.
FR-6	Редагування параметрів мінігри. Розробник змінює метадані та налаштування власної мінігри (опис, ознака преміум-доступу, локалізовані назви, шлях до ресурсів) через маніфест plugin.json.
FR-7	Перезавантаження web API серверу. Розробник ініціює перезапуск серверного застосунку, під час якого PluginLoader повторно підвантажує плагіни та застосовує додавання, оновлення чи вилучення мініігор.
FR-8	Поповнення балансу та платіжні операції. Сукупність функцій сервісу Payment, що забезпечують поповнення внутрішнього балансу через криптовалютний переказ USDT та супровід платіжних операцій.
FR-9	Налаштування адрес гаманців у середовищі (.env). Адміністратор задає адреси гаманців для мереж BEP20 та TRC20, які платформа використовує під час створення замовлень на поповнення.
FR-10	Налаштування доступу до платіжного API (.env). Адміністратор задає ключ і секрет Binance API, що використовуються для підпису HMAC-SHA256 запитів під час перевірки депозитів.
FR-11	Поповнення балансу через переказ USDT. Користувач обирає пакет монет і мережу, отримує унікальну суму та адресу гаманця, виконує переказ; після автоматичної перевірки депозиту через Binance API баланс поповнюється, а неоплачене за 30 хвилин замовлення скасовується.

Продовження таблиці 2.19

Назва	Опис
FR-12	Перегляд історії транзакцій. Адміністратор переглядає перелік замовлень на поповнення та операцій маркету з їх статусами (Pending, Completed, Expired), сумами та мережами з можливістю фільтрації.
FR-13	Ручне підтвердження транзакції. Адміністратор вручну зараховує кошти за замовленням, не підтвердженим автоматично, виконуючи транзакційне оновлення статусу замовлення та балансу користувача.
FR-14	Адміністрування та моніторинг платформи. Сукупність функцій контролю за роботою платформи, керування користувачами, контентом та конфігурацією.
FR-15	Перегляд логів системи. Адміністратор переглядає журнали подій платформи з можливістю фільтрації за рівнем та періодом.
FR-16	Перегляд статистики платформи. Адміністратор переглядає зведені показники роботи платформи за обраний період.
FR-17	Управління користувачами. Адміністратор переглядає облікові записи, блокує, розблоковує та змінює ролі користувачів.
FR-18	Управління преміум-підписками. Адміністратор активує, продовжує або скасовує преміум-підписки користувачів.
FR-19	Адміністрування маркету. Адміністратор керує асортиментом, цінами та видимістю предметів внутрішньоігрового маркету.
FR-20	Адміністрування рейтингу. Адміністратор налаштовує правила нарахування рейтингу та коригує рейтингові записи.

FR \ UC	UC-01	UC-02	UC-03	UC-04	UC-05	UC-06	UC-07	UC-08	UC-09	UC-10	UC-11	UC-12	UC-13	UC-14	UC-15	UC-16	UC-17
FR-1	+	+	+	+	+	+											
FR-2	+																
FR-3						+											
FR-4		+															
FR-5			+														
FR-6				+													
FR-7					+												
FR-8							+	+							+	+	+
FR-9							+										
FR-10								+									
FR-11																	+
FR-12															+		
FR-13																+	
FR-14									+	+	+	+	+	+			
FR-15									+								
FR-16										+							
FR-17											+						
FR-18												+					
FR-19													+				
FR-20														+			

Рисунок 2.1 – Матриця трасування вимог

2.3 Розроблення нефункціональних вимог

Передбачено вертикальне та горизонтальне масштабування. Вертикальне досягається нарощуванням ресурсів окремого сервера – обсягу оперативної пам'яті, кількості ядер та дискового простору – і дозволяє підвищити пропускну здатність авторизаційних запитів і вибірок з

рейтингової таблиці без архітектурних змін. Горизонтальне реалізується розподілом серверних компонентів – веб-сервера ASP.NET Core[12] та СУБД PostgreSQL[13] – на кілька вузлів із балансуванням навантаження та реплікацією бази даних для запитів читання [14], що дає змогу обслуговувати більшу кількість одночасних входів і переглядів сторінок рейтингу.

Доступність реалізованої частини має бути не нижче 99,9 %, що відповідає допустимому часу простою у 8 годин 45 хвилин на рік. Середній час між відмовами (MTBF) [15] має становити не менше 720 годин безперервної роботи, а середній час відновлення (MTTR) після збою не повинен перевищувати 30 хвилин завдяки автоматичному перезапуску застосунку, моніторингу стану ключових компонентів, наявності резервних серверних вузлів та регулярному резервному копіюванню бази даних.

Облікові записи користувачів, баланс внутрішньої грошової валюти, агрегований рейтинг гравців та історія операцій із преміум-доступом зберігаються у базі даних безстроково – до моменту явного видалення облікового запису користувачем через сторінку профілю. Журнали подій сервісу облікових записів, що фіксують факти входів, реєстрацій, змін пароля, скидань та нарахування місячної премії переможцю категорії, утримуються 6 місяців із наступним перенесенням до архіву та зберіганням ще протягом 18 місяців у режимі лише для читання. Щоденне резервне копіювання бази даних з утриманням резервних копій протягом трьох місяців забезпечує можливість відновлення облікових записів та рейтингової інформації на будь-яку дату цього періоду.

Конфігурованість реалізованої частини забезпечується винесенням ключових параметрів у файл конфігурації застосунку без модифікації коду. До таких параметрів належать правила складності пароля та мінімально допустима довжина нікнейму, терміни дії сесії користувача й токена відновлення пароля, періодичність роботи фонових завдань щомісячного перерахунку лідерів, перелік ролей, що виключаються з участі у нагородженні, формула обчислення розміру призу переможця місяця, а також параметри

інтеграції із зовнішнім провайдером Google для входу. Зміна будь-якого з перелічених значень не потребує перезбирання застосунку та виконується редагуванням файлу конфігурації з подальшим перезапуском сервісу.

2.4 Постановка завдання на розробку програмного забезпечення

Необхідно розробити підсистему динамічної інтеграції сторонніх ігрових модулів у вигляді плагінів та сервіс поповнення внутрішнього балансу для онлайн-платформи мініігор Quizanchos. Підсистема інтеграції має забезпечувати підключення, реєстрацію, публікацію та виведення з експлуатації окремих мініігор без модифікації ядра платформи: кожна мінігра постачається як окрема DLL-збірка, що реалізує визначений програмний інтерфейс і самостійно містить ігрову логіку, правила та клієнтські ресурси, тоді як платформа надає їй готову інфраструктуру – реєстри мініігор, маршрутизацію запитів і статичну роздачу ресурсів. Сервіс поповнення балансу має забезпечувати зарахування внутрішньої ігрової валюти через криптовалютний переказ USDT без залучення сторонньої процесингової платформи, з однозначним зіставленням депозитів із замовленнями користувачів.

Обидва компоненти реалізуються у складі серверного застосунку на ASP.NET Core з використанням Entity Framework Core [16] та СУБД PostgreSQL. Ізоляція сторонніх збірок одна від одної досягається механізмом AssemblyLoadContext із перенаправленням спільних із ядром типів, а взаємодія з платіжним шлюзом виконується підписаними HMAC-SHA256 запитам до Binance Sapi API з періодичною фоною обробкою засобами Quartz.NET.

Метою розробки є забезпечення стабільного та безпечного розширення платформи новими мініграми силами сторонніх розробників без зміни її ядра, а також надання користувачам надійного механізму поповнення внутрішнього балансу.

Розробка призначена для організації інтеграції зовнішніх мініігор та обробки поповнень внутрішнього балансу в межах єдиного серверного застосунку, а її кінцевими користувачами є: зовнішні розробники, що інтегрують власні мінігри у платформу через визначений програмний інтерфейс; авторизовані користувачі, що поповнюють внутрішній баланс; адміністратори, які конфігурують сервіси, ведуть моніторинг та опрацьовують спірні платіжні операції.

Розробка ставить перед собою досягнення наступних цілей:

- формалізація програмного інтерфейсу інтеграції мініігор у вигляді переносного SDK-пакета та динамічне підвантаження сторонніх DLL-збірок у власних ізольованих контекстах без впливу на ядро платформи;
- забезпечення реєстрації, публікації, оновлення та виведення з експлуатації мініігор у каталозі платформи з гарантією несуперечливості ідентифікаторів вбудованих і сторонніх мініігор;
- реалізація поповнення внутрішнього балансу через прямий прийом USDT у мережах BEP20 та TRC20 з однозначним зіставленням депозитів із замовленнями та автоматичним нарахуванням коштів;
- забезпечення адміністрування та конфігурування реалізованих сервісів, зокрема керування платіжними параметрами середовища, моніторинг роботи та ручне опрацювання спірних транзакцій.

Висновки до розділу

У другому розділі проведено аналіз вимог до програмного забезпечення підсистеми інтеграції сторонніх мініігор та сервісу поповнення внутрішнього балансу, у результаті чого сформовано технічне завдання на їх розробку.

У підрозділі 2.1 виокремлено та формалізовано сімнадцять варіантів використання, що охоплюють ключові сценарії інтеграції, адміністрування та поповнення балансу на платформі з боку трьох категорій акторів – розробника, адміністратора та авторизованого користувача. З боку розробника описано завантаження пакета мінігри, реєстрацію мінігри в каталозі, її публікацію для

гравців, редагування параметрів, перезавантаження web API серверу та підтримку NuGet-пакета інтерфейсу мінігри. З боку адміністратора – налаштування середовища (.env) для гаманця та платіжного API, перегляд логів і статистики платформи, управління користувачами та преміум-підписками, адміністрування маркету й рейтингу, перегляд історії транзакцій маркету та ручне підтвердження транзакції. З боку авторизованого користувача – поповнення балансу через криптовалютний переказ. Для кожного варіанта використання визначено цілі, передумови, основний потік подій, розширення та постумови, що забезпечує однозначне розуміння очікуваної поведінки системи.

У підрозділі 2.2 на основі варіантів використання розроблено набір з 20 функціональних вимог, згрупованих у три функціонально відокремлені модулі: інтеграція мініігор від розробника (сервіс PluginSystem), поповнення балансу та платіжні операції (сервіс Payment), а також адміністрування та моніторинг платформи. Для кожної вимоги визначено пріоритет та оцінку ризику реалізації, що дозволяє планувати поетапну розробку з урахуванням критичності функціоналу та технічної складності. Зв'язки між варіантами використання та функціональними вимогами представлено у вигляді матриці трасування, яка підтверджує повноту покриття сценаріїв використання функціональними вимогами.

У підрозділі 2.3 сформовано перелік нефункціональних вимог реалізованої частини, які охоплюють можливості вертикального та горизонтального масштабування, доступність на рівні не нижче 99,9 % із заданими показниками MTBF та MTTR, строки і порядок зберігання даних та резервного копіювання, а також конфігурованість ключових параметрів через файл конфігурації без зміни коду.

У підрозділі 2.4 на основі результатів аналізу сформульовано постановку завдання на розробку, у якій визначено мету, призначення, перелік кінцевих користувачів та основні цілі – формалізацію інтерфейсу інтеграції мініігор і динамічне підвантаження плагінів, реєстрацію та публікацію

мініігор, поповнення балансу через USDT та адміністрування реалізованих сервісів, що становлять основу для подальшого конструювання програмного забезпечення у наступних розділах роботи.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

Загальну архітектуру платформи Quizanchos (монолітний застосунок ASP.NET Core 10 з плагінним розширенням, контекстну та контейнерну діаграми моделі C4 [17]) описано у спільній частині, підрозділ 3.1. У межах індивідуальної частини детально розкривається два сервіси: PluginSystem (динамічне підвантаження сторонніх збірок мініігор) та Payment (поповнення внутрішнього балансу через криптовалютний шлюз). Обидва сервіси розташовані в каталозі Quizanchos.WebApi/Services і використовуються або з інших сервісів, або безпосередньо з фонових завдань Quartz.NET.

Компонентний рівень моделі C4 для зазначених двох сервісів подано на рисунку у графічних матеріалах “Компонентна діаграма сервісів PluginSystem та Payment (C4, рівень 3)”. На діаграмі відображено внутрішні компоненти обох сервісів та їх взаємодію із суміжними елементами застосунку: сервіс PluginSystem представлено компонентами PluginLoader, PluginLoadContext, PluginManifest, LoadedPlugin та IPluginCatalog, а сервіс Payment – компонентами TopUpService, BinanceDepositService, DepositCheckerJob і OrderExpiryJob. Показано також точку входу Startup, глобальні реєстри дескрипторів мініігор та зовнішні взаємодії з каталогом плагінів, Binance Sapi API і СУБД PostgreSQL.

Сервіс PluginSystem відповідає за виявлення та підвантаження сторонніх мініігор як окремих .NET-збірок у runtime. Точкою входу є статичний клас PluginLoader: за вказаним у конфігурації шляхом Plugins:Root він перераховує піддиректорії та намагається завантажити кожну як окремий плагін. Метадані плагіна (ім'я входного асемблі, шлях до wwwroot) описуються файлом plugin.json і десеріалізуються у тип PluginManifest. Для ізоляції збірок одна від одної кожна піддиректорія обгортається у власний PluginLoadContext – наслідник System.Runtime.Loader.AssemblyLoadContext з підключеним

AssemblyDependencyResolver. Усередині завантаженої збірки засобами рефлексії [18] шукаються класи, що реалізують IMinigameDescriptor та IMinigameFrontendDescriptor; знайдені дескриптори інстанціюються та реєструються у глобальних реєстрах IMinigameRegistry і IMinigameFrontendRegistry, які потім використовує сервіс GameLogic для маршрутизації запитів. Результат завантаження зберігається в об'єктах LoadedPlugin, що містять посилання на збірку, контекст та PhysicalFileProvider для wwwroot плагіна.

Сервіс Payment реалізує поповнення внутрішнього балансу через стейблкоїн USDT у мережах BEP20 та TRC20, без використання сторонніх процесингових платформ. TopUpService приймає запит від користувача на створення замовлення, обирає пакет із фіксованою кількістю монет за конфігурацією CoinPackageOption, генерує унікальну суму USDT (з точністю до останнього дробового розряду) та повертає клієнту адресу гаманця відповідної мережі. Періодичне фонове завдання DepositCheckerJob раз на N хвилин звертається до BinanceDepositService, який через підписаний HMAC-SHA256 виклик /sapi/v1/capital/deposit/hisrec отримує перелік нещодавніх депозитів. Збіг по парі (сума, мережа) дозволяє ідентифікувати, до якого саме TopUpOrder належить депозит, після чого замовлення переводиться у стан Completed і користувачеві нараховуються монети. OrderExpiryJob періодично скасовує замовлення, які не були оплачені протягом 30 хвилин з моменту створення.

3.2 Архітектурні рішення та обґрунтування вибору засобів розробки

У межах розробки сервісів PluginSystem та Payment прийнято низку рішень, спрямованих на можливість сторонньої розробки мініігор, безпечну ізоляцію їхнього коду у runtime та просту обробку платежів без процесингової платформи. Загальні рішення (мова, сервер застосунків, СКБД) узгоджено у спільному підрозділі 3.2; нижче розкривається лише та частина, яка стосується безпосередньо цих двох сервісів.

Для ізоляції плагінів обрано механізм `AssemblyLoadContext` (.NET 10) [19] замість окремих `AppDomain` (відсутні в сучасних версіях .NET) та замість процесів-співрозмовників (надмірна складність IPC для синхронної ігрової логіки). Кожна плагінна збірка підвантажується у власний `PluginLoadContext` з `AssemblyDependencyResolver`, що дозволяє кожному плагіну принести власні залежності NuGet-пакетів без конфліктів між версіями. Спільні з ядром збірки (`Quizanchos.Core`, `Quizanchos.Common` та абстракції DI) свідомо не дублюються – резолвер перенаправляє їх у контекст за замовчуванням, щоб типи `IMinigameDescriptor`, `IGameEngine` та `IGameLogic` були спільними між хостом і плагінами.

Для опису плагіна використано файл `plugin.json` (тип `PluginManifest`), а не конвенцію за назвою папки. Це дозволяє розробникам плагінів вільно називати свої збірки і вказувати власний шлях до `wwwroot`, не змінюючи структури каталогу. У разі відсутності файлу `PluginLoader` коректно повертається до значень за замовчуванням, припускаючи входним асемблї файл з іменем папки. Такий підхід зменшує бар'єр входу для перших плагінів та водночас не обмежує гнучкість пізніших.

Резервацію діапазону `MinigameTypeId` ≥ 1000 для сторонніх плагінів виконано на рівні самого `PluginLoader`: під час інстанціювання дескриптора перевіряється, що його `MinigameTypeId` не менший за поріг, інакше викидається виключення. Це гарантує, що третя сторона ніколи не зможе перевизначити вбудовані ідентифікатори вбудованих мініігор (`TypeId` 1, 3 тощо) і конфлікти у `IMinigameRegistry` виключені на старті, а не у `runtime`.

Як спосіб поповнення внутрішнього балансу обрано прямий прийом USDT у мережах BEP20 (Binance Smart Chain) та TRC20 (Tron) через гаманець на біржі Binance, замість інтеграції з процесинговою платформою (Stripe, Fondy, LiqPay). Це рішення мотивовано трьома причинами: відсутністю комісій процесора (Binance не стягує комісію за прийом депозитів), відсутністю вимог KYC до користувача платформи (на відміну від карткового процесора з 3-D Secure) і простотою аудиту – кожна транзакція має

детермінований TxId у блокчейні. Для ідентифікації, до якого замовлення належить конкретний депозит, використано схему унікальної суми (мінімальний дробовий розряд як зашумлювач) замість мемо: BEP20 та TRC20 USDT не підтримують мемо, тож унікальна сума – єдиний спосіб однозначно зіставити вхідний переказ із TopUpOrder без введення додаткових ідентифікаторів від користувача.

Перевірка надходження депозитів виконується періодичним опитуванням Binance Sapi API /sapi/v1/capital/deposit/hisrec у фоновому завданні DepositCheckerJob. Альтернативний варіант – webhook від Binance – наразі не передбачений політикою API, тож обрано pull-модель з інтервалом у кілька хвилин. Підпис кожного запиту виконується алгоритмом HMAC-SHA256 [20] з API-секретом, який зберігається у конфігурації BinanceApiOptions і передається у застосунок через змінні оточення/.env, ніколи не комітячись у репозиторій.

Для організації фонових завдань (DepositCheckerJob, OrderExpiryJob) обрано бібліотеку Quartz.NET [21]. Альтернативи – IHostedService із власним Timer або BackgroundService – відхилено через відсутність cron-розкладів, missfire-політики та персистентного збереження стану виконання. Quartz.NET дозволяє декларативно описати розклад «кожні N хвилин» без додаткового коду таймерів.

3.3 Конструювання програмного забезпечення

У межах конструювання сервісів PluginSystem та Payment розроблено два нетривіальні алгоритми [22]: динамічне підвантаження плагіна мінігри на старті застосунку та обробку циклу поповнення балансу через USDT. Решта операцій сервісів реалізовані як прямолінійні CRUD-методи та докладного опису у цьому підрозділі не потребують.

Алгоритм підвантаження плагіна виконується методом PluginLoader.LoadFromDirectory на старті ASP.NET Core застосунку і складається з наступних кроків.

- Перевіряється існування директорії `Plugins:Root` з конфігурації. Якщо її немає, повертається порожня колекція – старт застосунку не блокується відсутністю плагінів, що зручно для локальної розробки.
- Для кожної піддиректорії робиться спроба прочитати файл `plugin.json` і десеріалізувати його у тип `PluginManifest`. У разі відсутності або помилки парсингу використовуються значення за замовчуванням (`EntryAssembly = <FolderName>.dll`, `WwwRoot = «wwwroot»`). Помилка парсингу одного плагіна не зриває завантаження інших – виключення логується і обробка продовжується.
- Створюється екземпляр `PluginLoadContext` (наслідник `AssemblyLoadContext`) з конструктором, у який передаються шлях до входного асемблі та ім'я плагіна. Усередині контексту створюється `AssemblyDependencyResolver`, який відповідає за пошук залежних збірок у каталозі плагіна. Спільні з хостом збірки (`Quizanchos.Core`, `Quizanchos.Common`, абстракції DI) свідомо перенаправляються у контекст за замовчуванням, щоб уникнути конфліктів між різними версіями цих типів і дати плагіну змогу обмінюватися даними з ядром через спільні типи SDK.
- Завантажена збірка передається у допоміжний метод `InstantiateAll<T>`, який засобами рефлексії перебирає `assembly.GetTypes()`, фільтрує не-абстрактні класи, що реалізують `T` (`IMinigameDescriptor` або `IMinigameFrontendDescriptor`), і створює їх через `Activator.CreateInstance`. Помилка завантаження окремого типу (`ReflectionTypeLoadException`) обробляється акуратно: до результату потрапляють лише ті типи, які вдалося завантажити, а решта пропускається з логуванням.
- Для кожного отриманого `IMinigameDescriptor` виконується додаткова перевірка `MinigameTypeId ≥ 1000`. Якщо хоч один дескриптор порушує цю інваріанту, плагін відхиляється повністю – виключення гарантує, що жоден сторонній плагін не може перевизначити вбудовані ідентифікатори мініігор.

– Якщо у каталозі плагіна існує піддиректорія `wwwroot`, для неї створюється `PhysicalFileProvider`, який монтується у статичний шлях `/minigames/{gamekey-lowercase}/` у `Startup`. Це дозволяє клієнтському браузеру завантажувати CSS- та JS-ресурси конкретного плагіна без перебудови основного фронтенду.

– Усі дані про завантажений плагін (`PluginId`, шлях до збірки, посилання на `Assembly` та `PluginLoadContext`, списки дескрипторів, `FileProvider`) акумулюються в об'єкті `LoadedPlugin` і додаються до колекції результату. Подальша реєстрація у `IMinigameRegistry` та `IMinigameFrontendRegistry` виконується у `Startup` за допомогою методу `RegisterServices` самого дескриптора.

Алгоритм поповнення балансу охоплює два сценарії: створення замовлення на стороні застосунку та виявлення оплати у блокчейні через `Binance API`. Обидва сценарії координуються сервісом `TopUpService` у поєднанні з фоновими завданнями.

– Користувач обирає пакет монет та мережу (`BER20` чи `TRC20`) і викликає `TopUpService.CreateOrderAsync`. Сервіс валідує параметри (мережа з білого списку, існування пакета), отримує `ApplicationUserId` через `UserRetrieverService` та обчислює унікальну суму `USDT` через `GenerateUniqueAmountAsync`: до базової ціни пакета додається випадковий зашумлювач у останніх дробових розрядах, унікальність якого перевіряється запитом до `ITopUpOrderRepository`. Створений `TopUpOrder` зберігається зі статусом `Pending` і часом створення `CreatedAtUtc`.

– Клієнту повертається `CreateTopUpOrderResultDto` з адресою гаманця обраної мережі (`BinanceApiOptions.UsdtAddressBer20` або `UsdtAddressTrc20`), точною сумою для переказу та часом дії замовлення (30 хвилин від створення).

– Фонове завдання `DepositCheckerJob` (`Quartz.NET` cron-розклад «кожні N хвилин») викликає `BinanceDepositService.GetRecentDepositsAsync(coin, network, startTime)`.

Сервіс будує QueryString, обчислює HMAC-SHA256 підпис з API-секретом, додає заголовок X-MBX-APIKEY та виконує GET-запит до /sapi/v1/capital/deposit/hisrec. У разі помилки HTTP запит логується, але не зриває роботу завдання – наступний цикл повторить опитування.

- Отриманий список депозитів зіставляється з активними TopUpOrder у статусі Pending за парою (AmountUSDT, Network). Якщо знайдено збіг, TopUpService.ConfirmOrderAsync виконує транзакцію EF: статус замовлення переводиться у Completed, у ApplicationUser.Coins додається кількість монет із пакета (CoinsToCredit), у журналі фіксується TxId. Транзакція гарантує, що часткове застосування неможливе.

- Окреме фонове завдання OrderExpiryJob періодично перебирає замовлення у статусі Pending, для яких різниця CreatedAtUtc від поточного UTC-моменту перевищує 30 хвилин, і переводить їх у статус Expired. Це звільняє унікальні суми для повторного використання та закриває некоректно покинуті замовлення.

3.4 Опис структури бази даних

Сервіси PluginSystem та Payment працюють з невеликою підмножиною таблиць загальної схеми PostgreSQL [23]. Повний перелік таблиць наведений у спільному підрозділі 3.4; нижче перераховані лише ті, що використовуються двома розглянутими сервісами.

- top_up_order – таблиця активних і завершених замовлень на поповнення. Містить ApplicationUserId, статус (Pending, Completed, Expired), кількість монет CoinsToCredit, точну суму AmountUSDT, мережу (BEP20 чи TRC20) та час створення CreatedAtUtc. Усі звернення виконуються через ITopUpOrderRepository; унікальність комбінації AmountUSDT та Network серед Pending-замовлень забезпечує можливість однозначного зіставлення депозиту з замовленням.

- AspNetUsers (ApplicationUser) – таблиця облікових записів ASP.NET Core Identity. Сервіс Payment звертається до неї лише для оновлення

поля Coins під час підтвердження замовлення; повний опис таблиці та інших полів наведено в індивідуальній частині співавтора (у складі сервісу Users).

– `game_type` – каталог зареєстрованих типів мініігор. Сервіс `PluginSystem` не пише у цю таблицю безпосередньо – рядки створюються через міграції або фікстури під час старту, але `PluginSystem` використовує дані таблиці для зіставлення завантажених `IMinigameDescriptor` з рядком `game_type` за `gameKey`.

Сторонні плагіни мініігор, незважаючи на динамічне підвантаження їх збірок у runtime, не отримують прямого доступу до бази даних: усі звернення до `GameSessionState` виконує хост-реалізація `IGameStatePersistence` (описана у частині співавтора). Завдяки цьому жодна таблиця, окрім переліченої вище, плагінами не використовується безпосередньо, що мінімізує атакувальну поверхню сторонніх збірок.

Висновки до розділу

У третьому розділі індивідуальної частини детально описано архітектуру та конструювання двох сервісів серверного застосунку `Quizanchos.WebApi`, розробкою яких займався автор: `PluginSystem`, що динамічно підвантажує сторонні збірки мініігор через `PluginLoader` та `PluginLoadContext` на базі `AssemblyLoadContext`; `Payment`, що реалізує прямий прийом USDT у мережах BEP20 та TRC20 через `TopUpService` та `DepositCheckerJob` із підписаними HMAC-SHA256 запитами до `Binance Sapi`. Для цих сервісів побудовано окрему компонентну діаграму C4 рівня 3.

Обґрунтовано ключові архітектурні рішення: використання `AssemblyLoadContext` з `AssemblyDependencyResolver` та перенаправленням спільних залежностей до контексту за замовчуванням; маніфест `plugin.json` як гнучкий опис плагіна; резервацію діапазону `MinigameTypeId` ≥ 1000 для сторонніх плагінів; прийом USDT через біржовий гаманець `Binance` замість процесингової платформи з ідентифікацією за унікальною сумою; pull-модель опитування `Binance Sapi API` підписаними HMAC-SHA256 запитами;

використання Quartz.NET для фонових завдань з cron-розкладами. На рівні конструювання детально розкрито алгоритм підвантаження плагіна та цикл поповнення балансу через USDT. Решта компонентів WebApi (Rooms, GameLogic, Market, Users) описана в індивідуальній частині співавтора та у спільному розділі.

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Опис процесів тестування

Перевірку правильності реалізації функціоналу, розробленого автором, – сервісів динамічного підвантаження сторонніх мініігор (PluginSystem) та поповнення внутрішнього балансу через криптовалюту USDT (Payment) – здійснено за допомогою автоматизованого модульного тестування на базі фреймворка xUnit.net[24].

Автоматизовану перевірку винесено у відокремлений тестовий проєкт Quizanchos.WebApi.Tests, що містить 31 тест, згрупований у п'ять тестових класів. Тести побудовано за класичним шаблоном Arrange–Act–Assert на базі фреймворка xUnit.net 2.9 [25] та цільового середовища .NET 10. Тестовану логіку перевірено ізолювано: жоден тест не звертається до бази даних, до реального API біржі Binance чи до завантаження сторонніх збірок у runtime. Залежність від бази даних замінено легкою реалізацією репозиторію [26] ITopUpOrderRepository в оперативній пам'яті; криптографічне підписування запитів та правила ізоляції плагінів перевіряються як детерміновані чисті функції; а сценарії сканування каталогу плагінів використовують тимчасову директорію, що видаляється після прогону. Така побудова забезпечує детермінованість прогону та незалежність результату від інфраструктурного оточення розробника.

До групи тестів підсистеми підвантаження плагінів (PluginSystem) увійшли три тестові класи, що сукупно містять 18 тестів. PluginLoaderTests (4 тести) перевіряє стійкість сканування кореневого каталогу плагінів: відсутній або порожній каталог, папку плагіна без входного асемблі та пошкоджений файл-маніфест plugin.json система має толерувати без винятків, повертаючи порожній результат і не блокуючи старт застосунку. PluginLoadContextTests (10 тестів) верифікує правило розподілу збірок між хостом та ізольованим контекстом завантаження: контрактні збірки SDK (Quizanchos.Core,

Quizanchos.Common) та збірки платформи (Microsoft.*, System.*, netstandard, mscorlib) делегуються контексту за замовчуванням задля рекурсивної рівності типів дескрипторів між ядром і плагінами, тоді як власні сторонні залежності плагіна завантажуються в його ізольованому контексті. PluginManifestTests (4 тести) перевіряє коректність десеріалізації маніфесту plugin.json, у тому числі нечутливість до реєстру імен полів та коректну обробку відсутніх полів, для яких далі застосовуються значення за замовчуванням.

До групи тестів сервісу поповнення балансу (Payment) увійшли два тестові класи, що сукупно містять 13 тестів. BinanceSignatureTests (4 тести) перевіряє підписування запитів до Binance Sapi алгоритмом HMAC-SHA256: відповідність обчисленого підпису незалежно сформованому контрольному вектору, детермінованість, залежність підпису від секретного ключа та формат результату (64 символи у нижньому реєстрі шістнадцяткової системи). TopUpServiceTests (9 тестів) покриває бізнес-логіку створення замовлення на поповнення: відображення каталогу пакетів монет у DTO, валідацію мережі переказу за білим списком (BEP20, TRC20) та відхилення невідомого пакета, а також алгоритм генерування унікальної суми USDT – перевіряється, що згенерована сума перевищує базову ціну пакета на зашумлювач у останньому дробовому розряді, що сума попередньо перевіряється на відсутність серед активних замовлень і що у разі вичерпання діапазону унікальних сум сервіс повертає контрольовану помилку.

Запуск усіх автоматизованих тестів виконується стандартною командою dotnet test [27] для проєкту Quizanchos.WebApi.Tests. Усі 31 тестових сценаріїв проходять успішно, без помилок та пропусків. Підсумковий результат проходження тестів наведено на рисунку 4.1.

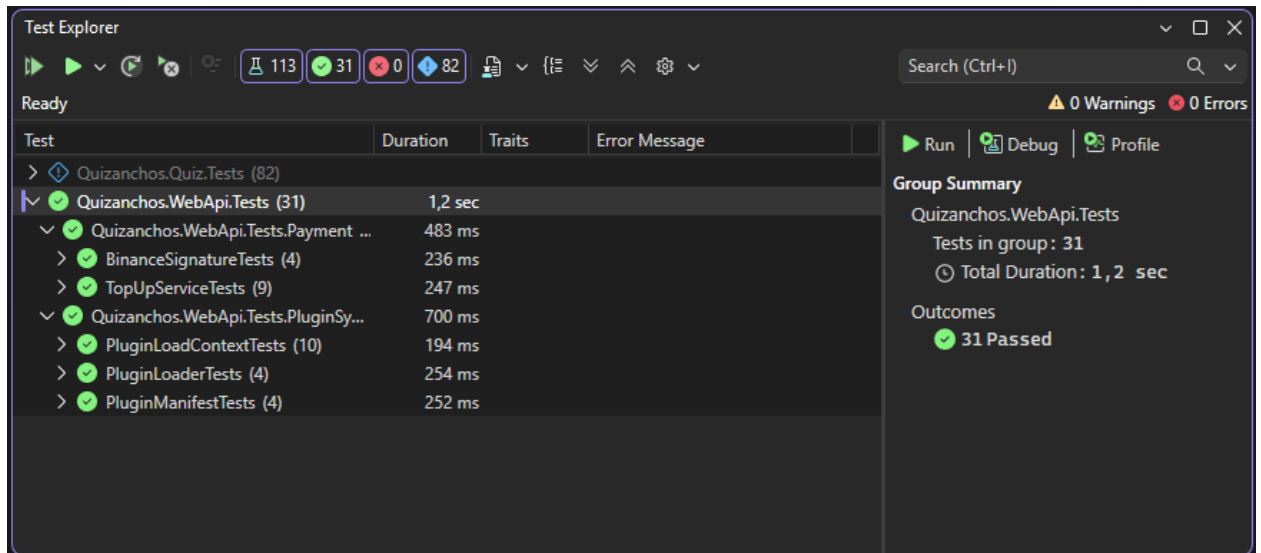


Рисунок 4.1 – Результати успішного проходження автоматизованих тестів

Окрім автоматизованого модульного тестування, для перевірки реалізованої функціональності в умовах, наближених до реального використання платформи, виконано серію мануальних тестів [28, 29]. Кожен тест охоплює окремий сценарій взаємодії із сервісами індивідуальної частини – динамічним підключенням сторонніх мініігор (PluginSystem) та поповненням внутрішнього балансу криптовалютою USDT (Payment). Результати тестування наведено у таблицях 4.1 – 4.12.

Таблиця 4.1 – Тест 1. Підключення коректного стороннього плагіна до каталогу мініігор

Початковий стан системи	Серверний застосунок зупинено; кореневий каталог плагінів, заданий параметром конфігурації Plugins:Root, містить лише раніше підключені плагіни; каталог мініігор платформи налічує N доступних ігор.
Вхідні дані	Папка плагіна зі скомпільованою збіркою мінігри, підкаталогом wwwroot та коректним файлом-маніфестом plugin.json (поля entryAssembly та wwwRoot); клас-дескриптор має MinigameTypeId = 1000.
Опис проведення тесту	Папку плагіна копіюють до каталогу Plugins:Root та перезавантажують серверний застосунок.

Продовження таблиці 4.1

Очікуваний результат	Під час старту PluginLoader завантажує збірку в окремий PluginLoadContext, засобами рефлексії знаходить класи-дескриптори та реєструє мінігру у реєстрах IMinigameRegistry і IMinigameFrontendRegistry; у журналі застосунку фіксується успішне завантаження плагіна із зазначенням його ідентифікатора; каталог мініігор містить N+1 гру.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.2 – Тест 2. Проходження новопідключеної мінігри у браузері

Початковий стан системи	Плагін із тесту 1 успішно завантажено; користувач авторизований і перебуває в каталозі мініігор.
Вхідні дані	Ключ (GameKey) новопідключеної сторонньої мінігри.
Опис проведення тесту	У браузері користувач відкриває нову мінігру з каталогу та проходить ігрову партію.
Очікуваний результат	Клієнтський застосунок завантажує статичні ресурси мінігри за маршрутом /minigames/{ключ-мінігри}/, змонтованим власним постачальником файлів PhysicalFileProvider; ігрова логіка виконується через спільні контракти SDK, стан сесії зберігається через надану ядром реалізацію IGameStatePersistence; партія завершується коректним підсумком.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.3 – Тест 3. Толерування пошкодженого файлу-маніфесту plugin.json

Початковий стан системи	Серверний застосунок зупинено; у каталозі Plugins:Root поряд з валідними плагінами розміщено папку плагіна із синтаксично некоректним файлом plugin.json.
Вхідні дані	Папка плагіна з пошкодженим (нечитабельним) маніфестом plugin.json.
Опис проведення тесту	Перезавантаження застосунку зі змішаним набором коректних і пошкодженого плагінів.
Очікуваний результат	PluginLoader пропускає проблемний плагін без генерування винятку, старт застосунку не блокується, решта плагінів завантажуються штатно; у журналі фіксується попередження про пропущений плагін.

Продовження таблиці 4.3

Фактичний результат	Збігається з очікуванням.
---------------------	---------------------------

Таблиця 4.4 – Тест 4. Папка плагіна без вхідного асемблі

Початковий стан системи	Серверний застосунок зупинено; у каталозі Plugins:Root присутня папка плагіна, що не містить вхідного асемблі.
Вхідні дані	Папка плагіна без збірки, зазначеної у полі entryAssembly.
Опис проведення тесту	Перезавантаження серверного застосунку.
Очікуваний результат	PluginLoader толерує таку папку, повертає для неї порожній результат сканування й пропускає плагін без винятку; старт застосунку завершується успішно, інші плагіни завантажуються.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.5 – Тест 5. Відхилення плагіна із зарезервованим ідентифікатором MinigameTypeId < 1000

Початковий стан системи	Серверний застосунок зупинено; у каталозі Plugins:Root розміщено плагін, клас-дескриптор якого оголошує MinigameTypeId = 1 (діапазон вбудованих мініігор).
Вхідні дані	Клас-дескриптор мінігри зі значенням MinigameTypeId = 1.
Опис проведення тесту	Перезавантаження серверного застосунку.
Очікуваний результат	Під час реєстрації порушено інваріант MinigameTypeId ≥ 1000 , тож мінігра не реєструється у глобальних реєстрах і не потрапляє до каталогу; конфлікту з ідентифікаторами вбудованих мініігор не виникає; решта плагінів завантажуються коректно.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.6 – Тест 6. Ізоляція власних залежностей плагінів

Початковий стан системи	Серверний застосунок зупинено; у каталозі Plugins:Root підключено два плагіни, кожен з яких має у власній папці свою версію однієї і тієї ж сторонньої бібліотеки.
-------------------------	--

Продовження таблиці 4.6

Вхідні дані	Дві збірки плагінів із різними версіями спільної сторонньої залежності.
Опис проведення тесту	Перезавантаження серверного застосунку та послідовне проходження обох мініігор у браузері.
Очікуваний результат	Кожен плагін завантажує власну версію сторонньої залежності у своєму PluginLoadContext, тоді як контрактні збірки SDK (Quizanchos.Core, Quizanchos.Common) та збірки платформи делегуються контексту за замовчуванням; обидві мінігри працюють без конфлікту версій.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.7 – Тест 7. Створення замовлення на поповнення балансу через мережу BEP20

Початковий стан системи	Авторизований користувач перебуває на сторінці поповнення балансу; його поточний баланс становить В монет.
Вхідні дані	Пакет монет із каталогу (наприклад, «1000 монет»); мережа переказу BEP20.
Опис проведення тесту	Користувач обирає пакет монет і мережу BEP20 та підтверджує намір поповнити баланс.
Очікуваний результат	TopUpService створює замовлення на поповнення, генерує унікальну суму USDT (базова ціна пакета, збільшена на зашумлювач у останньому дробовому розряді) та повертає адресу гаманця разом із сумою для переказу; у репозиторії з'являється активне замовлення у статусі очікування оплати.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.8 – Тест 8. Зарахування коштів після підтвердженого депозиту

Початковий стан системи	Існує активне замовлення з тесту 7; на адресу гаманця переказано точну унікальну суму USDT, транзакцію підтверджено в мережі.
Вхідні дані	Депозит на унікальну суму, що відповідає активному замовленню.
Опис проведення тесту	Спрацьовує фонове завдання DepositCheckerJob, яке через BinanceDepositService опитує історію депозитів і зіставляє надходження із замовленням.

Продовження таблиці 4.8

Очікуваний результат	Замовлення переходить у статус виконаного, баланс користувача збільшується на кількість монет придбаного пакета (стає В + пакет); повторне зарахування того самого депозиту не відбувається.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.9 – Тест 9. Унікальність суми за наявності активного замовлення на той самий пакет

Початковий стан системи	Існує активне (ще не оплачене) замовлення на пакет «1000 монет».
Вхідні дані	Повторний запит на поповнення тим самим пакетом монет і тією самою мережею.
Опис проведення тесту	Користувач створює друге замовлення на той самий пакет, поки перше залишається активним.
Очікуваний результат	TopUpService попередньо перевіряє згенеровану суму на відсутність серед активних замовлень і формує іншу унікальну суму USDT, відмінну від суми першого замовлення, що забезпечує однозначне зіставлення майбутніх депозитів із замовленнями.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.10 – Тест 10. Прострочення замовлення на поповнення балансу

Початковий стан системи	Існує активне замовлення на поповнення, з моменту створення якого минув відведений час життя (TTL), а депозит не надійшов.
Вхідні дані	Активне замовлення без отриманого депозиту.
Опис проведення тесту	Спрацьовує фонове завдання OrderExpiryJob, що обробляє прострочені замовлення.
Очікуваний результат	Замовлення переходить у статус простроченого, зарезервована за ним унікальна сума звільняється для повторного використання іншими замовленнями, баланс користувача залишається незмінним.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.11 – Тест 11. Відхилення запиту з невідомим пакетом поповнення

Початковий стан системи	Авторизований користувач перебуває на сторінці поповнення балансу.
Вхідні дані	Ідентифікатор пакета монет, відсутнього у каталозі пакетів.

Продовження таблиці 4.11

Опис проведення тесту	Надсилається запит на створення замовлення з невідомим ідентифікатором пакета.
Очікуваний результат	TopUpService відхиляє запит контрольованою помилкою; замовлення на поповнення не створюється, баланс користувача не змінюється.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.12 – Тест 12. Відхилення неприпустимої мережі переказу

Початковий стан системи	Авторизований користувач перебуває на сторінці поповнення балансу.
Вхідні дані	Коректний пакет монет; мережа переказу, відсутня у білому списку (наприклад, ERC20).
Опис проведення тесту	Користувач намагається створити замовлення з мережею, яка не входить до білого списку підтримуваних мереж (BER20, TRC20).
Очікуваний результат	Валідація мережі за білим списком відхиляє запит; замовлення не створюється, користувач отримує повідомлення про перелік підтримуваних мереж переказу.
Фактичний результат	Збігається з очікуваним.

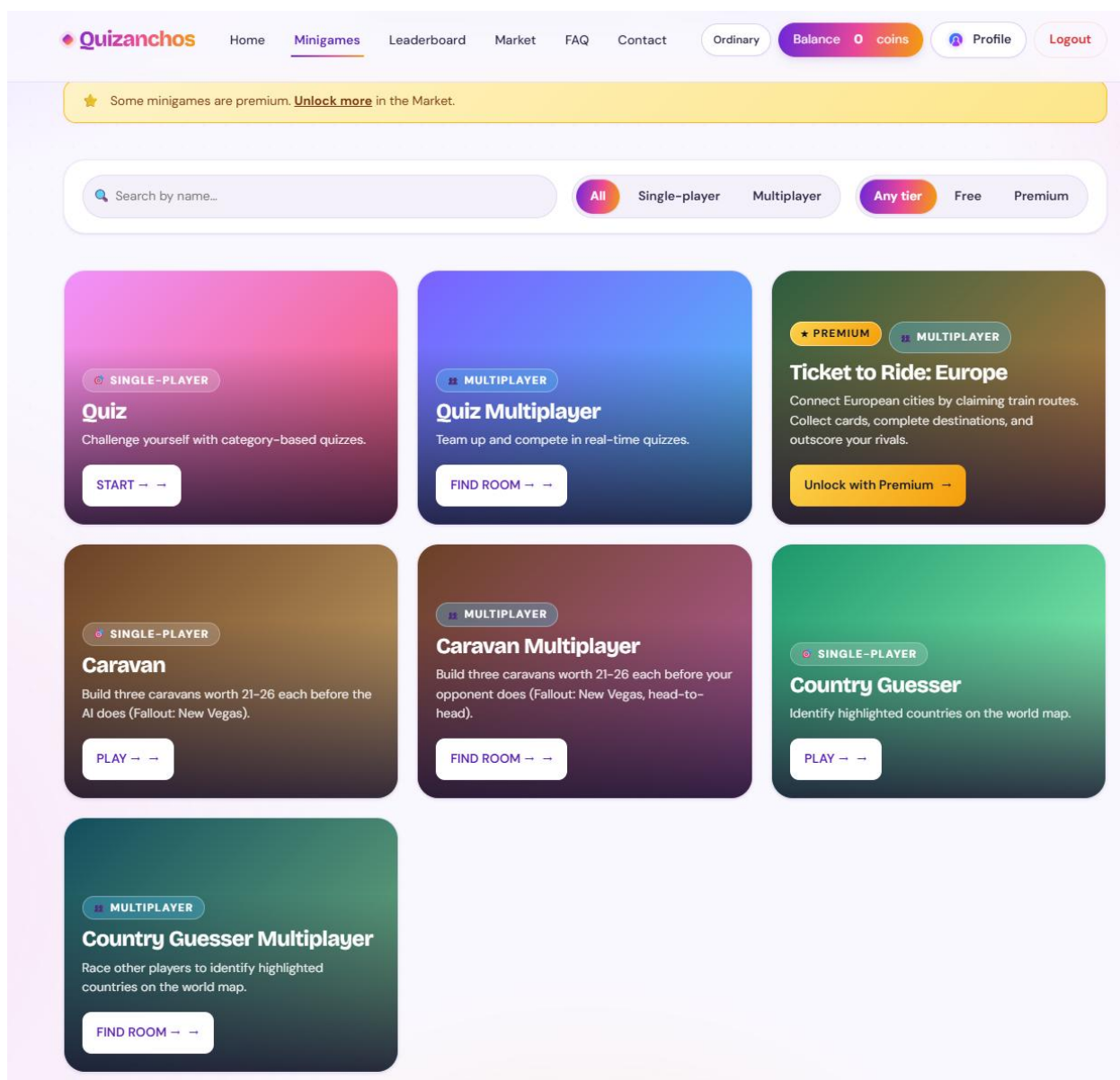
Перелічених у таблицях 4.1 – 4.12 дванадцять мануальних тестів достатньо для покриття основних сценаріїв роботи сервісів PluginSystem та Payment індивідуальної частини. Сценарії сервісу Payment виконано вручну у браузері Chromium 132, а сценарії підключення плагінів – шляхом розміщення підготовлених збірок у каталозі плагінів, перезавантаження серверного застосунку та контролю результату в журналі застосунку і каталозі мініігор. Результати кожного з тестів збігаються з очікуваними.

4.2 Опис контрольного прикладу

Контрольний приклад демонструє ключову можливість розробленого автором сервісу PluginSystem – динамічне підключення нової сторонньої мінігри до платформи без зміни та перекомпіляції коду ядра. Сценарій побудовано навколо переходу системи з початкового стану, у якому каталог

містить N мініігор, до кінцевого стану з $N+1$ мінігрою після того, як розробник підготує та підключить новий плагін і перезавантажить серверний застосунок.

У початковому стані каталог мініігор платформи містить N доступних ігор – вбудовані мінігри Quiz і Quiz Multiplayer та раніше підключені сторонні плагіни. Саме цей перелік бачить користувач до підключення нової гри (рисуюнок 4.2).



Рисуюнок 4.2 – Початковий стан каталогу мініігор (N мініігор)

Підключення нової мінігри виконується розробником у кілька кроків. Перший крок – створення окремого проєкту класової бібліотеки .NET, до якого підключається лише SDK-пакет Quizanchos.Core (пакет Quizanchos.Common підтягується транзитивно); посилання на проєкти ядра

Quizanchos.WebApi чи Quizanchos.Domain заборонені. У цьому проєкті розробник реалізує три складові: клас-дескриптор бекенду на основі IMinigameDescriptor (визначає ключ гри GameKey, ідентифікатор MinigameTypeId, тип ходу та створення ігрового рушія), клас-дескриптор фронтенду на основі IMinigameFrontendDescriptor (перелік CSS/JS-ресурсів і метадані відображення) та власне ігрову логіку на основі IGameLogic. Обов'язковою умовою є присвоєння MinigameTypeId значення не меншого за 1000 – діапазону, зарезервованого для сторонніх плагінів, що унеможливлює конфлікт із ідентифікаторами вбудованих мініігор (рисунок 4.3).

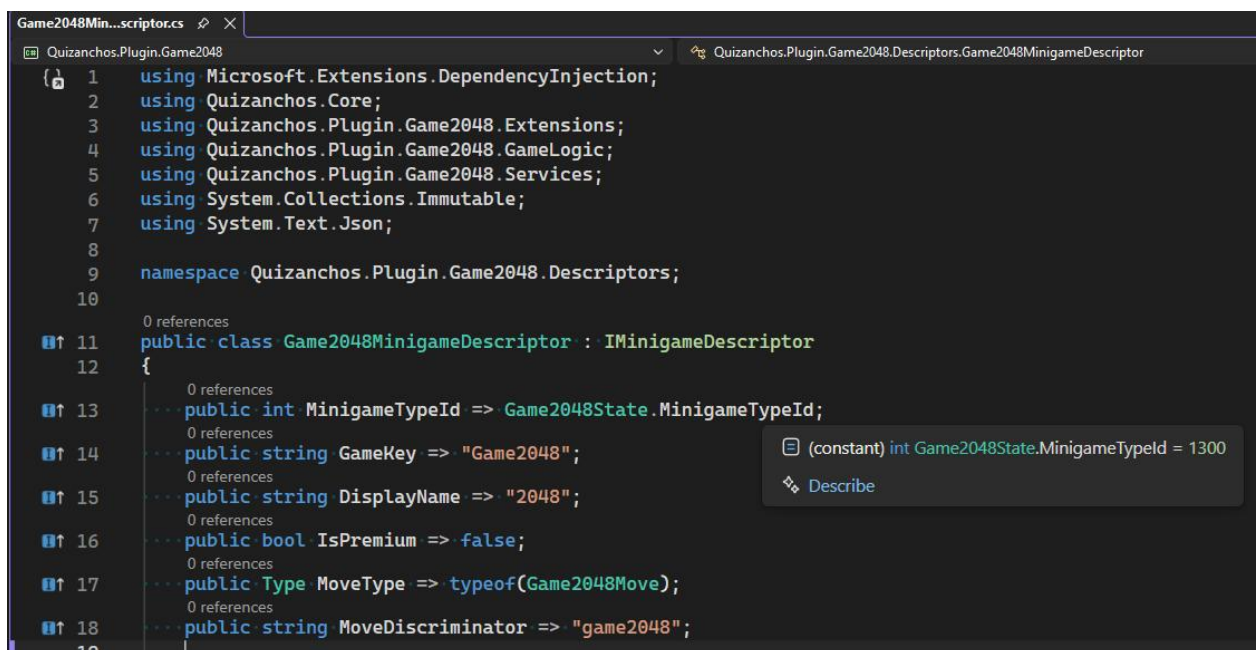


Рисунок 4.3 – Клас-дескриптор нової мінігри з $\text{MinigameTypeId} \geq 1000$

Другий крок – складання та розміщення плагіна. Розробник компілює проєкт командою `dotnet publish`, унаслідок чого отримує збірку мінігри разом з її залежностями та підкаталогом `wwwroot` із клієнтськими файлами. Поряд зі збіркою він розміщує файл-маніфест `plugin.json`, у якому зазначає ім'я вхідного асемблі (поле `entryAssembly`) та шлях до статичних ресурсів (поле `wwwRoot`); за відсутності маніфесту `PluginLoader` застосовує значення за замовчуванням, виведені з імені папки. Підготовлену в такий спосіб папку плагіна розробник копіює до кореневого каталогу плагінів, заданого параметром конфігурації `Plugins:Root` (рисунок 4.4).

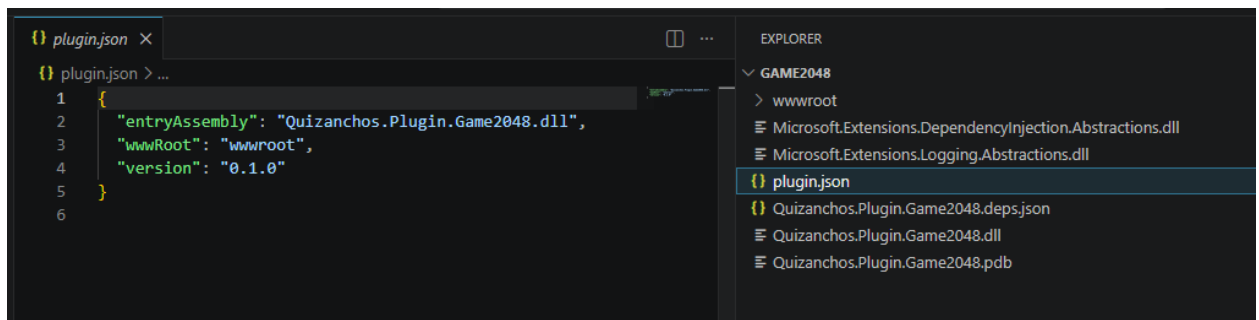


Рисунок 4.4 – Структура каталогу плагіна та вміст файлу-маніфесту plugin.json

Третій крок – перезавантаження серверного застосунку, щоб він підхопив нову мінігру. Під час старту клас PluginLoader сканує кореневий каталог плагінів, завантажує кожну збірку в ізолюваний контекст PluginLoadContext, засобами рефлексії знаходить класи-дескриптори, перевіряє інваріанту $\text{MinigameTypeId} \geq 1000$ та реєструє мінігру у глобальних реєстрах IMinigameRegistry і IMinigameFrontendRegistry. Помилка завантаження одного плагіна не зриває старту інших, а успішне завантаження нового плагіна фіксується у журналі застосунку із зазначенням його ідентифікатора та кількості зареєстрованих дескрипторів (рисунок 4.5).

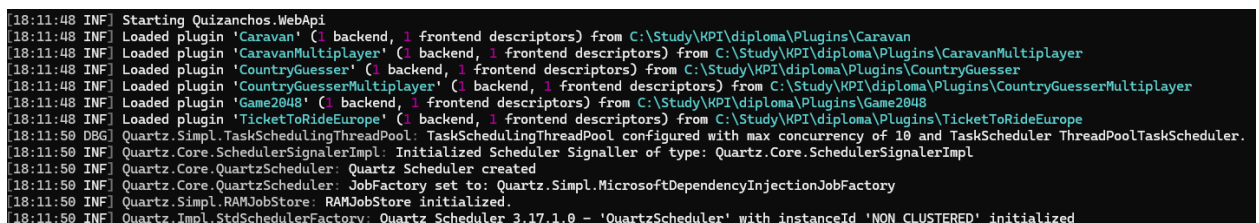


Рисунок 4.5 – Журнал застосунку: завантаження нового плагіна під час перезавантаження

Унаслідок виконаних кроків система переходить у кінцевий стан: каталог мінігор містить уже $N+1$ гру, де новопідключена мінігра відображається нарівні з рештою. При цьому жоден рядок коду ядра платформи не було змінено чи перекомпільовано – розширення відбулося виключно за рахунок додавання нової збірки до каталогу плагінів (рисунок 4.6).

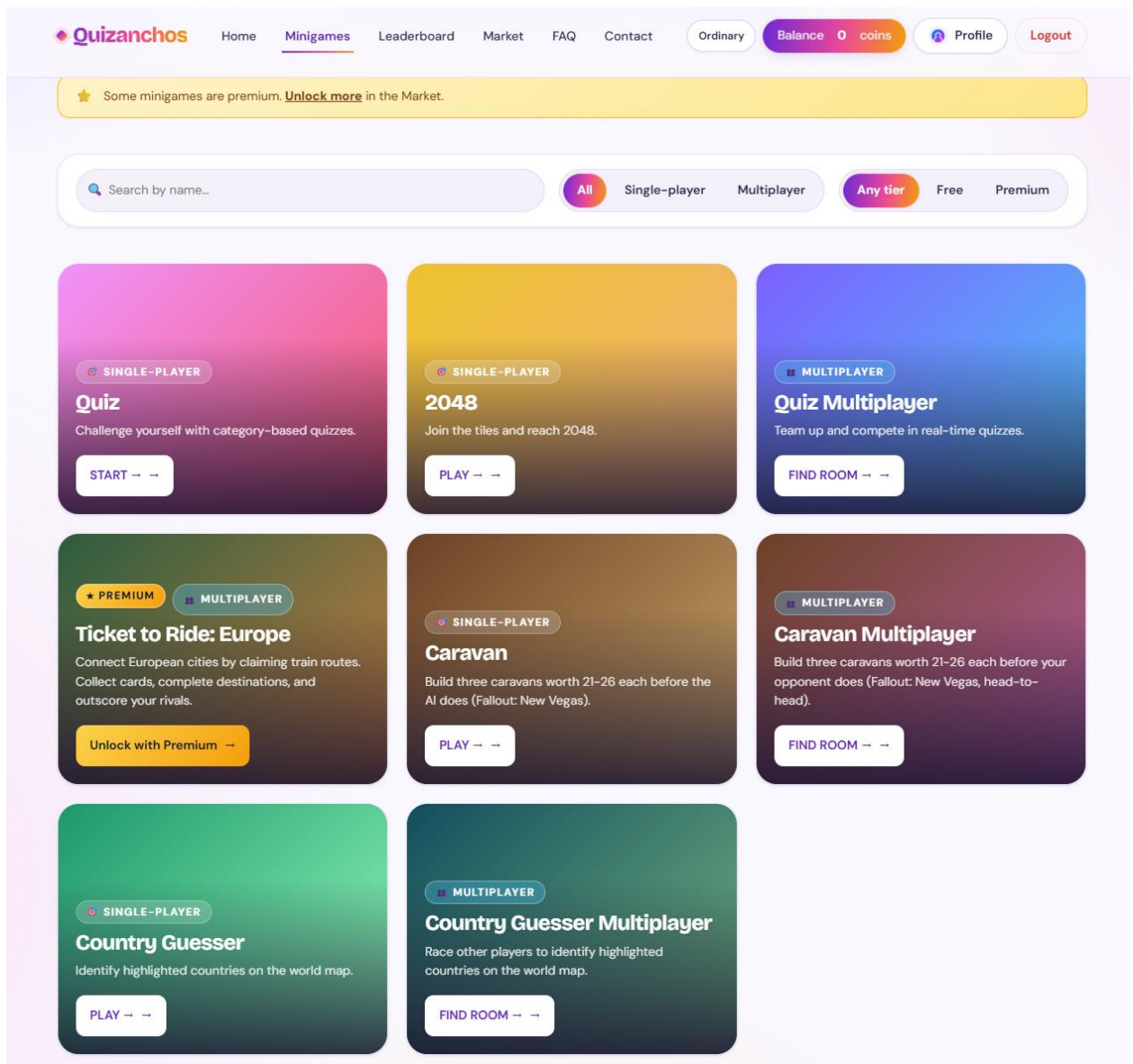


Рисунок 4.6 – Кінцевий стан каталогу мініігор (N+1 мініігра)

Для остаточної перевірки інтеграції користувач відкриває новопідключену мінігру. Клієнтський застосунок завантажує її статичні ресурси за маршрутом `/minigames/{ключ-мінігри}/`, змонтованим для плагіна власним постачальником файлів `PhysicalFileProvider`, а ігрова логіка виконується через спільні контракти SDK зі збереженням стану сесії через надану ядром реалізацію `IGameStatePersistence`. Успішне проходження партії підтверджує повну та коректну інтеграцію нової мінігри (рисунок 4.7).

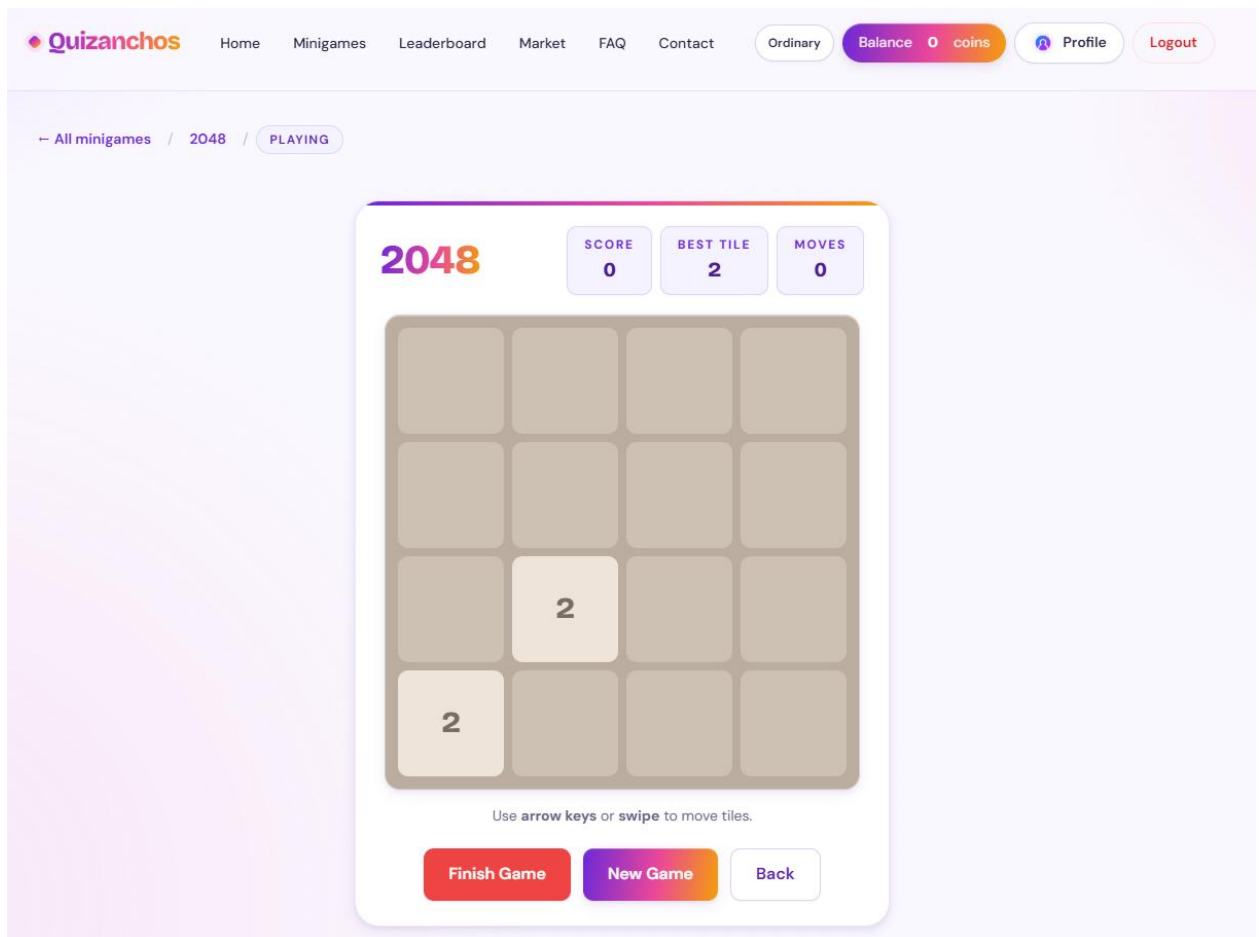


Рисунок 4.7 – Проходження новопідключеної мінігри

Розглянутий контрольний приклад послідовно демонструє повний життєвий цикл інтеграції сторонньої мінігри засобами сервісу PluginSystem: від початкового стану з N мініграми, через підготовку розробником нового плагіна (реалізація дескрипторів та ігрової логіки, складання, опис у `plugin.json` і розміщення у каталозі плагінів) та перезавантаження застосунку, до кінцевого стану з $N+1$ мінігрою. Це підтверджує, що платформа дозволяє розширювати перелік мініігор без зміни коду ядра, а поведінка розробленого програмного забезпечення відповідає сформульованим у розділі 2 вимогам.

Висновки до розділу

У четвертому розділі виконано аналіз якості [30] та перевірку правильності реалізації двох розроблених автором сервісів серверного застосунку – PluginSystem і Payment. Основним інструментом верифікації обрано автоматизоване модульне тестування на базі фреймворка xUnit.net,

винесене у відокремлений тестовий проєкт Quizanchos.WebApi.Tests, у якому розроблено 31 тест, згрупований у п'ять тестових класів за шаблоном Arrange–Act–Assert. Тести підсистеми підвантаження плагінів охоплюють стійкість сканування каталогу плагінів до некоректних вхідних даних, правило ізоляції збірок між хостом і плагіном та десеріалізацію маніфесту plugin.json; тести сервісу поповнення балансу – підписування запитів до Binance алгоритмом HMAC-SHA256, валідацію мережі й пакета поповнення та генерування унікальної суми USDT для однозначного зіставлення депозиту із замовленням. Тести побудовано детермінованими та незалежними від інфраструктурного оточення; усі 31 тестових сценаріїв проходять успішно, без помилок та пропусків.

Окрім автоматизованого модульного тестування, проведено серію з дванадцяти мануальних тестів, що відтворюють роботу сервісів PluginSystem і Payment в умовах, наближених до реального використання платформи. Сценарії сервісу Payment – створення замовлення на поповнення через мережу BEP20, зарахування коштів після підтвердженого депозиту, забезпечення унікальності суми за наявності активного замовлення, прострочення замовлення та відхилення запитів з невідомим пакетом чи неприпустимою мережею переказу – виконано вручну у браузері Chromium. Сценарії сервісу PluginSystem – підключення коректного стороннього плагіна та проходження новопідключеної мінігри, толерування пошкодженого маніфесту plugin.json і папки без вхідного асемблі, відхилення плагіна із зарезервованим ідентифікатором MinigameTypeId < 1000 та ізоляцію власних залежностей плагінів – перевірено шляхом розміщення підготовлених збірок у каталозі плагінів, перезавантаження серверного застосунку та контролю результату в журналі застосунку й каталозі мініігор. Результати всіх дванадцяти тестів збігаються з очікуваними.

За допомогою контрольного прикладу продемонстровано повний життєвий цикл динамічного підключення сторонньої мінігри засобами сервісу PluginSystem – від початкового стану каталогу з N мініграми, через підготовку

розробником нового плагіна (реалізацію дескрипторів та ігрової логіки на основі SDK-пакета Quizanchos.Core, складання збірки, опис у plugin.json і розміщення у каталозі плагінів) та перезавантаження серверного застосунку, до кінцевого стану з N+1 мінігрою. Підключення нової гри відбувається без зміни чи перекомпіляції коду ядра платформи, що підтверджує закладену в архітектуру вимогу до розширюваності. Сукупність отриманих результатів – повне та безпомилкове проходження автоматизованих тестів, успішне виконання всіх мануальних тестів і відтворений сценарій контрольного прикладу – свідчить про те, що розроблені сервіси PluginSystem та Payment відповідають сформульованим у розділі 2 функціональним і нефункціональним вимогам та готові до розгортання та подальшого супроводу.

ВИСНОВКИ

У ході виконання дипломного проєкту розроблено веб-платформу Quizanchos, що об'єднує в єдиному середовищі каталог мініігор, мультиплеєрну взаємодію в реальному часі та систему обігу віртуальних активів. Платформа реалізована на стеку ASP.NET Core 10 та PostgreSQL за плагінною архітектурою, у межах якої окремі мінігри підключаються до ядра як незалежні DLL-модулі.

Поставлену мету щодо забезпечення стабільної та ефективної роботи платформи шляхом організації взаємодії між програмними інтерфейсами мініігор та інтерфейсом платформи досягнуто у повному обсязі. Успішне проходження автоматизованих тестів та послідовне виконання сценаріїв контрольного прикладу, наведені у четвертому розділі, підтверджують відповідність розробленого програмного забезпечення усім сформульованим функціональним та нефункціональним вимогам.

Першу з поставлених функціональних задач, що стосувалася формалізації програмного інтерфейсу інтеграції мініігор, розв'язано шляхом створення переносного SDK-пакета Quizanchos.Core. До нього увійшли базові інтерфейси IMinigameDescriptor, IGameLogic та IGameStatePersistence, які описують контракт між мінігрою і ядром платформи. Завдяки цьому сторонній розробник має можливість підключити власну мінігру у вигляді DLL-збірки без жодного втручання у код ядра.

Другу задачу, що передбачала динамічне підключення сторонніх мініігор до платформи без зміни її ядра, реалізовано засобами підсистеми PluginSystem. Кожна мінігра підвантажується з каталогу плагінів у власному ізольованому контексті PluginLoadContext на базі AssemblyLoadContext, а статичний клас PluginLoader засобами рефлексії виявляє та реєструє її дескриптори у глобальних реєстрах IMinigameRegistry і IMinigameFrontendRegistry з контролем зарезервованого діапазону ідентифікаторів MinigameTypeId ≥ 1000 .

Третя задача стосувалася надання користувачам надійного способу поповнення внутрішнього балансу і вирішена шляхом реалізації сервісу Payment. Поповнення виконується прямим прийомом криптовалюти USDT у мережах BEP20 та TRC20 через біржовий гаманець Binance: сервіс TopUpService створює замовлення з унікальною сумою для однозначного зіставлення платежу, а фонові завдання Quartz.NET виявляють підтверджені депозити через Binance Sapi API й автоматично нараховують кошти в межах транзакції бази даних.

Розроблена платформа є придатною для подальшої експлуатації, супроводу та розширення новими мініігровими модулями. Це підтверджує доцільність продовження розробки за відповідною предметною областю, зокрема у напрямі підключення додаткових ігрових жанрів, удосконалення рейтингових механізмів та інтеграції з новими платіжними шлюзами.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Chou Yu-kai. Actionable Gamification: Beyond Points, Badges, and Leaderboards. Packt Publishing, Limited, 2019.
- 2) Clements P., Bass L., Kazman R. Software Architecture in Practice. Pearson Education, Limited, 2021.
- 3) Create a .NET Core application with plugins - .NET. Microsoft Learn: Build with answers in reach. URL: <https://learn.microsoft.com/en-us/dotnet/core/tutorials/creating-app-with-plugin-support> (дата звернення: 12.03.2026).
- 4) Lehdonvirta V., Castronova E. Virtual Economies: Design and Analysis. The MIT Press, 2014.
- 5) Freund J., Rücker B. Real-Life BPMN: Includes an Introduction to DMN. 4th ed. Camunda, 2019.
- 6) Antonopoulos A. M., Wood G. Mastering Ethereum: Building Smart Contracts and DApps. O'Reilly Media, 2018.
- 7) Antonopoulos A. M. Mastering Bitcoin: Programming the Open Blockchain. 2nd ed. O'Reilly Media, 2017.
- 8) Deposit History (supporting network). Binance API Documentation. URL: <https://developers.binance.com/docs/wallet/capital/deposite-history> (дата звернення: 27.04.2026).
- 9) Bashir I. Mastering Blockchain. 3rd ed. Packt Publishing, Limited, 2020.
- 10) Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- 11) Wiegers K., Beatty J. Software Requirements. 3rd ed. Microsoft Press, 2013.
- 12) Esposito D. Programming ASP. NET Core, Programming ASP. NET Core. Microsoft Press, 2018.
- 13) Vannahme A., Juba S., Volkov A. Learning PostgreSQL. Packt Publishing, Limited, 2015.

- 14) Kleppmann M. Designing Data-Intensive Applications. O'Reilly Media, 2017.
- 15) Beyer B., Jones C., Petoff J., Murphy N. R. Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, 2016.
- 16) Smith J. P. Entity Framework Core in Action. 2nd ed. Manning Publications, 2021.
- 17) Brown S. The C4 model for visualising software architecture. URL: <https://c4model.com> (дата звернення: 05.03.2026).
- 18) Richter J. CLR via C#. 4th ed. Microsoft Press, 2012.
- 19) AssemblyLoadContext Class (System.Runtime.Loader). Microsoft Learn: Build with answers in reach. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.runtime.loader.assemblyloadcontext> (дата звернення: 19.05.2026).
- 20) Krawczyk H., Bellare M., Canetti R. HMAC: Keyed-Hashing for Message Authentication. RFC 2104. URL: <https://www.rfc-editor.org/rfc/rfc2104> (дата звернення: 08.04.2026).
- 21) Quartz.NET Documentation. Quartz Enterprise Scheduler .NET. URL: <https://www.quartz-scheduler.net/documentation/> (дата звернення: 23.03.2026).
- 22) McConnell S. Code Complete: A Practical Handbook of Software Construction. 2nd ed. Microsoft Press, 2004.
- 23) PostgreSQL 17 Documentation. The PostgreSQL Global Development Group. URL: <https://www.postgresql.org/docs/current/> (дата звернення: 15.05.2026).
- 24) Unit testing C# code in .NET using dotnet test and xUnit - .NET. Microsoft Learn: Build with answers in reach. URL: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-csharp-with-xunit> (дата звернення: 02.04.2026).
- 25) Oshero R. The Art of Unit Testing. 2nd ed. Manning Publications, 2013.
- 26) Meszaros G. xUnit Test Patterns: Refactoring Test Code. Addison-Wesley, 2007.

- 27) Unit testing best practices for .NET. Microsoft Learn: Build with answers in reach. URL: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices> (дата звернення: 11.05.2026).
- 28) Graham D., van Veenendaal E., Evans I., Black R. Foundations of Software Testing: ISTQB Certification. Cengage Learning, 2008.
- 29) Myers G. J., Sandler C., Badgett T. The Art of Software Testing. 3rd ed. John Wiley & Sons, 2011.
- 30) ДСТУ ISO/IEC 25010:2016. Інженерія систем і програмного забезпечення. Вимоги до якості систем і програмного забезпечення та її оцінювання (SQuaRE). Моделі якості систем і програмних засобів. Київ : ДП «УкрНДНЦ», 2016.

ДОДАТОК А ЗВІТ ПОДІБНОСТІ



Документ прийнятий

Метадані

ДОКУМЕНТ

Заголовок

ІП-21_Павлусенко_ПЗ

Автор

ІП-21_Павлусенко

Науковий керівник / Експерт

Павлов О.А.

ІД документу

334108820

ОРГАНІЗАЦІЯ

Назва організації

National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute

підрозділ

ФІОТ, К-а інформатики та програмної інженерії

ЗВІТ

Дата звіту

5/31/2026

Дата редагування

6/1/2026

Обсяг знайдених подібностей

Коефіцієнт подібності визначає, який відсоток тексту по відношенню до загального обсягу тексту було знайдено в різних джерелах. Зверніть увагу, що високі значення коефіцієнта не автоматично означають плагіат. Звіт має аналізувати компетентна / уповноважена особа.



10392

Кількість слів

83348

Кількість символів

Тривога

У цьому розділі ви знайдете інформацію щодо текстових спотворень. Ці спотворення в тексті можуть говорити про МОЖЛИВІ маніпуляції в тексті. Спотворення в тексті можуть мати навмисний характер, але частіше характер технічних помилок при конвертації документа та його збереженні, тому ми рекомендуємо вам підходити до аналізу цього модуля відповідально. У разі виникнення запитань, просимо звертатися до нашої служби підтримки.

Заміна букв	Б	0
Інтервали	A→	0
Мікропробіли	␣	0
Білі знаки	␣	0
Парафрази (SmartMarks)	a	28

Джерела

Нижче наведений список джерел. В цьому списку є джерела із різних баз даних. Колір тексту означає в якому джерелі він був знайдений. Ці джерела і значення Коефіцієнту Подібності не відображають прямого плагіату. Необхідно відкрити кожне джерело і проаналізувати зміст і правильність оформлення джерела.

10 найдовших фраз		Колір тексту
#	НАЗВА ТА АДРЕСА ДЖЕРЕЛА URL (НАЗВА БАЗИ)	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	https://ela.kni.ua/bitstream/123456789/53536/1/Smurhok_hakalavr.pdf	28 (0.27 %)

2	https://ela.kpi.ua/bitstreams/e9cda8e5-c3a6-498a-993d-4ceff6f6c78a/download	19 (0.18 %)
3	ІП-11_Веремчук_Головатюк_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	16 (0.15 %)
4	ІП-311_Маслова_ПЗ 6/3/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	16 (0.15 %)
5	https://ela.kpi.ua/bitstreams/e9cda8e5-c3a6-498a-993d-4ceff6f6c78a/download	15 (0.14 %)
6	https://ela.kpi.ua/bitstreams/e9cda8e5-c3a6-498a-993d-4ceff6f6c78a/download	15 (0.14 %)
7	https://ela.kpi.ua/bitstreams/e9cda8e5-c3a6-498a-993d-4ceff6f6c78a/download	14 (0.13 %)
8	Вебзастосунок для громадських організацій наукових спільнот 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	13 (0.13 %)
9	Диплом Нурбек_к2_сдача.docx 6/5/2022 Kazakh-Deutsch University (Факультет инжиниринга и информационных технологий)	12 (0.12 %)
10	ІП-11_Лошак_ПЗ 6/3/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	11 (0.11 %)

Домашня база даних (3.02 %)



#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	ІП-11_Веремчук_Головатюк_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	61 (7) (0.59 %)
2	Вебзастосунок для громадських організацій наукових спільнот 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	55 (7) (0.53 %)
3	ІП-11_Лошак_ПЗ 6/3/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	49 (6) (0.47 %)
4	ІП-11_Веремчук_ПЗ 6/9/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	34 (5) (0.33 %)
5	ІП-11_Тихонов_ПЗ 5/31/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	34 (5) (0.33 %)

6	ІП-15_Легеза_ПЗ 6/6/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	31 (4) (0.3 %)
7	Платформа для аналізу та проєктування топологічних організацій розподілених систем 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	23 (3) (0.22 %)
8	ІП-311_Маслова_ПЗ 6/3/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	22 (2) (0.21 %)
9	Освітній веб-застосунок для оптимізації роботи логопедів та психологів 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	5 (1) (0.05 %)
Програма обміну базами даних (0.12 %)		
#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
10	Диплом Нурбек_к2 здача.docx 6/5/2022 Kazakh-Deutsch University (Факультет інжиниринга и информационных технологий)	12 (1) (0.12 %)
Інтернет (1.42 %)		
#	ДЖЕРЕЛО URL	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
11	https://ela.kpi.ua/bitstreams/e9cda8e5-c3a6-498a-993d-4ceff6f6c78a/download	100 (9) (0.96 %)
12	https://ela.kpi.ua/bitstream/123456789/53536/1/Smuchok_bakalavr.pdf	37 (2) (0.36 %)
13	https://www.scribd.com/document/867240931/Bilyi-bakalavr	11 (2) (0.11 %)
Список прийнятих фрагментів		
#	ЗМІСТ	КІЛЬКІСТЬ ОДНАКОВИХ СЛІВ (ФРАГМЕНТІВ)

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ОНЛАЙН ПЛАТФОРМИ МІНІГОР З
ПІДТРИМКОЮ МУЛЬТИПЛЕЄРНОЇ ВЗАЄМОДІЇ ТА СИСТЕМИ
ВІРТУАЛЬНИХ АКТИВІВ**

Текст програми

КПІ.ПІ-2116.045440.03.12

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

_____ Максим

ГОЛОВЧЕНКО

Виконавці:

_____ Данило ПАВЛУСЕНКО

Київ – 2026

Посилання на репозиторій з повним текстом програмного коду
<https://github.com/navarti/Quizanchos>

Файл IMinigameDescriptor.cs (Quizanchos.Core)

Реалізація функціональної задачі – формалізація програмного інтерфейсу інтеграції мінігри-плагіна з ядром платформи (декларує спосіб самореєстрації плагіна – реєстрацію сервісів, створення ігрового рушія та збереження стану партії).

```
using Microsoft.Extensions.DependencyInjection;
using System.Collections.Immutable;

namespace Quizanchos.Core;

/// <summary>
/// Descriptor for a minigame plugin. Allows self-registration of minigames
/// without modifying the WebApi code.
/// </summary>
public interface IMinigameDescriptor
{
    /// <summary>
    /// Numeric minigame identifier used by API contracts and persistence.
    /// </summary>
    int MinigameTypeId { get; }

    /// <summary>
    /// Unique identifier for the minigame (e.g., "Quiz", "Game2048", "QuizMultiplayer").
    /// </summary>
    string GameKey { get; }

    /// <summary>
    /// Human-readable display name
    /// </summary>
    string DisplayName { get; }

    /// <summary>
    /// Indicates whether this minigame requires premium access.
    /// </summary>
    bool IsPremium { get; }

    /// <summary>
    /// Concrete move type for this minigame used for polymorphic serialization.
    /// Must derive from <see cref="GameMove"/>.
    /// </summary>
    Type MoveType { get; }

    /// <summary>
    /// Type discriminator value used in JSON payloads for this minigame move type.
    /// </summary>
    string MoveDiscriminator { get; }

    /// <summary>
    /// Register this minigame's services with the DI container.
    /// This method is called during application startup.
    /// </summary>
    void RegisterServices(IServiceCollection services);

    /// <summary>
    /// Create a game engine for this minigame type.
    /// </summary>
    /// <param name="gameId">The unique identifier for the game instance</param>
    /// <param name="playerIds">Array of player IDs participating in the game</param>
    /// <param name="parameters">Game-specific parameters (e.g., difficulty, board size,
    etc.)</param>
    /// <param name="serviceProvider">Service provider for resolving dependencies</param>
    /// <returns>A new game engine instance</returns>
    Task<IGameEngine> CreateGameEngineAsync(Guid gameId, ImmutableArray<string> playerIds,
        Dictionary<string, object> parameters, IServiceProvider serviceProvider);

    /// <summary>
```



```

    /// Load a previously saved game engine from storage.
    /// </summary>
    /// <param name="gameId">The unique identifier for the game instance</param>
    /// <param name="serviceProvider">Service provider for resolving dependencies</param>
    /// <returns>The loaded game engine, or null if not found</returns>
    Task<IGameEngine?> LoadGameEngineAsync(Guid gameId, IServiceProvider serviceProvider);

    /// <summary>
    /// Save game state for persistence.
    /// </summary>
    /// <param name="gameId">The unique identifier for the game instance</param>
    /// <param name="state">The game state to persist</param>
    /// <param name="serviceProvider">Service provider for resolving dependencies</param>
    Task SaveGameStateAsync(Guid gameId, IGameState state, IServiceProvider serviceProvider);
}

```

Файл PluginLoader.cs (Quizanchos.WebApi/Services/PluginSystem)

Реалізація функціональної задачі – динамічне виявлення та завантаження зборок мініігор-плагінів у власних ізольованих контекстах AssemblyLoadContext під час старту застосунку.

```

using Microsoft.Extensions.FileProviders;
using Quizanchos.Core;
using System.Reflection;
using System.Text.Json;

namespace Quizanchos.WebApi.Services.PluginSystem;

/// <summary>
/// Scans a plugin root directory and loads each subdirectory as an isolated plugin.
/// </summary>
internal static class PluginLoader
{
    private const int MinThirdPartyMinigameTypeId = 1000;

    public static IReadOnlyList<LoadedPlugin> LoadFromDirectory(string pluginRoot, ILogger logger)
    {
        if (!Directory.Exists(pluginRoot))
        {
            logger.LogInformation("Plugin root '{Root}' does not exist; no third-party plugins loaded.", pluginRoot);
            return Array.Empty<LoadedPlugin>();
        }

        var loaded = new List<LoadedPlugin>();
        foreach (var pluginDir in Directory.EnumerateDirectories(pluginRoot))
        {
            var folderName = Path.GetFileName(pluginDir);
            try
            {
                var plugin = LoadOne(pluginDir, folderName, logger);
                if (plugin is not null)
                {
                    loaded.Add(plugin);
                    logger.LogInformation(
                        "Loaded plugin '{PluginId}' ({DescriptorCount} backend, {FrontendCount} frontend descriptors) from {Path}",
                        plugin.PluginId, plugin.Descriptors.Count,
                        plugin.FrontendDescriptors.Count, pluginDir);
                }
            }
            catch (Exception ex)
            {
                logger.LogError(ex, "Failed to load plugin from {Path}", pluginDir);
            }
        }

        return loaded;
    }

    private static LoadedPlugin? LoadOne(string pluginDir, string folderName, ILogger logger)
    {
        var manifest = ReadManifest(pluginDir, logger);
        var entryName = manifest?.EntryAssembly ?? $"{folderName}.dll";
    }
}

```

```

var entryPath = Path.Combine(pluginDir, entryName);

if (!File.Exists(entryPath))
{
    logger.LogWarning("Plugin folder '{Folder}' has no entry assembly '{Entry}'",
folderName, entryName);
    return null;
}

var loadContext = new PluginLoadContext(entryPath, folderName);
var assembly = loadContext.LoadFromAssemblyPath(entryPath);

var descriptors = InstantiateAll<IMinigameDescriptor>(assembly);
var frontendDescriptors = InstantiateAll<IMinigameFrontendDescriptor>(assembly);

if (descriptors.Count == 0 && frontendDescriptors.Count == 0)
{
    logger.LogWarning("Plugin '{Folder}' loaded but contained no descriptors",
folderName);
}

foreach (var descriptor in descriptors)
{
    if (descriptor.MinigameTypeId < MinThirdPartyMinigameTypeId)
    {
        throw new InvalidOperationException(
            $"Third-party plugin '{folderName}' descriptor
'{descriptor.GetType().FullName}' " +
            $"uses MinigameTypeId={descriptor.MinigameTypeId}, but third-party IDs must
be >= {MinThirdPartyMinigameTypeId}." );
    }
}

var wwwRootSubpath = manifest?.WwwRoot ?? "wwwroot";
var wwwRootFull = Path.Combine(pluginDir, wwwRootSubpath);
IFileProvider? fileProvider = Directory.Exists(wwwRootFull)
    ? new PhysicalFileProvider(wwwRootFull)
    : null;

return new LoadedPlugin
{
    PluginId = folderName,
    SourceDirectory = pluginDir,
    Assembly = assembly,
    LoadContext = loadContext,
    Descriptors = descriptors,
    FrontendDescriptors = frontendDescriptors,
    StaticFiles = fileProvider,
};
}

private static PluginManifest? ReadManifest(string pluginDir, ILogger logger)
{
    var manifestPath = Path.Combine(pluginDir, "plugin.json");
    if (!File.Exists(manifestPath))
    {
        return null;
    }

    try
    {
        var json = File.ReadAllText(manifestPath);
        return JsonSerializer.Deserialize<PluginManifest>(json,
            new JsonSerializerOptions { PropertyNameCaseInsensitive = true });
    }
    catch (Exception ex)
    {
        logger.LogWarning(ex, "Failed to read plugin manifest at {Path}; using defaults",
manifestPath);
        return null;
    }
}

private static IReadOnlyList<T> InstantiateAll<T>(Assembly assembly) where T : class
{
    Type[] types;
    try
    {
        types = assembly.GetTypes();
    }
}

```

```

        catch (ReflectionTypeLoadException ex)
        {
            types = ex.Types.Where(t => t is not null).Cast<Type>().ToArray();
        }

        return types
            .Where(t => t is { IsInterface: false, IsAbstract: false } &&
                typeof(T).IsAssignableFrom(t))
            .Select(t => Activator.CreateInstance(t) as T)
            .Where(d => d is not null)
            .Select(d => d!)
            .ToArray();
    }
}

```

Файл PluginManifest.cs (Quizanchos.WebApi/Services/PluginSystem)

Реалізація функціональної задачі – опис необов’язкових метаданих плагіна (ім’я вхідної збірки, тека статичних ресурсів, версія), що зчитуються з файлу plugin.json у теці плагіна.

```

namespace Quizanchos.WebApi.Services.PluginSystem;

/// <summary>
/// Optional plugin metadata loaded from <c>plugin.json</c> in a plugin folder.
/// All fields are optional; sensible defaults are derived from the folder name when missing.
/// </summary>
internal sealed record PluginManifest
{
    /// <summary>File name of the entry assembly inside the plugin folder. Defaults to
    <c>{folderName}.dll</c>.</summary>
    public string? EntryAssembly { get; init; }

    /// <summary>Subfolder containing the plugin's static assets. Defaults to
    <c>wwwroot</c>.</summary>
    public string? WwwRoot { get; init; }

    /// <summary>Plugin author-declared version. Informational only.</summary>
    public string? Version { get; init; }
}

```

Файл PluginLoadContext.cs (Quizanchos.WebApi/Services/PluginSystem)

Реалізація функціональної задачі – ізоляція збірки плагіна у власному контексті завантаження AssemblyLoadContext з перенаправленням спільних із хостом збірок до контексту за замовчуванням задля еталонної рівності типів дескрипторів.

```

using System.Reflection;
using System.Runtime.Loader;

namespace Quizanchos.WebApi.Services.PluginSystem;

/// <summary>
/// Isolated <see cref="AssemblyLoadContext"/> for a single plugin. Resolves the plugin's
/// transitive dependencies via its .deps.json, but defers SDK contract assemblies and
/// framework assemblies to the default context so descriptor types are reference-equal
/// across the host and all plugins.
/// </summary>
internal sealed class PluginLoadContext : AssemblyLoadContext
{
    private readonly AssemblyDependencyResolver _resolver;

    public PluginLoadContext(string pluginEntryPath, string name)
        : base(name: name, isCollectible: false)
    {
        _resolver = new AssemblyDependencyResolver(pluginEntryPath);
    }

    protected override Assembly? Load(AssemblyName assemblyName)
    {
        if (assemblyName.Name is { } name && IsHostSharedAssembly(name))

```

```

    {
        return null;
    }

    var assemblyPath = _resolver.ResolveAssemblyToPath(assemblyName);
    return assemblyPath is not null ? LoadFromAssemblyPath(assemblyPath) : null;
}

protected override IntPtr LoadUnmanagedDll(string unmanagedDllName)
{
    var libraryPath = _resolver.ResolveUnmanagedDllToPath(unmanagedDllName);
    return libraryPath is not null ? LoadUnmanagedDllFromPath(libraryPath) : IntPtr.Zero;
}

private static bool IsHostSharedAssembly(string name)
{
    return name.StartsWith("Quizanchos.Core", StringComparison.OrdinalIgnoreCase)
        || name.StartsWith("Quizanchos.Common", StringComparison.OrdinalIgnoreCase)
        || name.StartsWith("Microsoft.", StringComparison.OrdinalIgnoreCase)
        || name.StartsWith("System.", StringComparison.OrdinalIgnoreCase)
        || name.Equals("netstandard", StringComparison.OrdinalIgnoreCase)
        || name.Equals("mscorlib", StringComparison.OrdinalIgnoreCase);
}
}

```

Файл LoadedPlugin.cs (Quizanchos.WebApi/Services/PluginSystem)

Реалізація функціональної задачі – незмінна модель успішно завантаженого плагіна, що утримує посилання на збірку, контекст завантаження, дескриптори мініігор та постачальник статичних файлів.

```

using Microsoft.Extensions.FileProviders;
using Quizanchos.Core;
using System.Reflection;
using System.Runtime.Loader;

namespace Quizanchos.WebApi.Services.PluginSystem;

/// <summary>
/// A plugin that has been successfully loaded from disk. The <see cref="LoadContext"/>
/// reference must be retained for the plugin's lifetime to prevent the assembly from
/// being unloaded.
/// </summary>
public sealed record LoadedPlugin
{
    public required string PluginId { get; init; }
    public required string SourceDirectory { get; init; }
    public required Assembly Assembly { get; init; }
    public required AssemblyLoadContext LoadContext { get; init; }
    public required IReadOnlyList<IMinigameDescriptor> Descriptors { get; init; }
    public required IReadOnlyList<IMinigameFrontendDescriptor> FrontendDescriptors { get; init; }
    public IFileProvider? StaticFiles { get; init; }
}

```

Файл IPluginCatalog.cs (Quizanchos.WebApi/Services/PluginSystem)

Реалізація функціональної задачі – контракт і реалізація реєстру-одинака завантажених плагінів для подальшого монтування їхніх статичних ресурсів у конвеєрі проміжного програмного забезпечення.

```

namespace Quizanchos.WebApi.Services.PluginSystem;

/// <summary>
/// Read-only catalog of plugins loaded from the plugin root at startup.
/// Registered as a singleton so middleware setup can mount each plugin's static files.
/// </summary>
public interface IPluginCatalog
{
    IReadOnlyList<LoadedPlugin> Plugins { get; }
}

internal sealed class PluginCatalog : IPluginCatalog
{
}

```

```

public PluginCatalog(IReadOnlyList<LoadedPlugin> plugins)
{
    Plugins = plugins;
}

public IReadOnlyList<LoadedPlugin> Plugins { get; }
}

```

Файл **BinanceApiOptions.cs (Quizanchos.WebApi/Options)**

Реалізація функціональної задачі – клас параметрів конфігурації інтеграції з Binance API – ключі доступу та адреси USDT-гаманців мереж BEP20 і TRC20.

```

namespace Quizanchos.WebApi.Options;

public class BinanceApiOptions
{
    public string ApiKey { get; set; } = string.Empty;
    public string ApiSecret { get; set; } = string.Empty;
    public string UsdtAddressBep20 { get; set; } = string.Empty;
    public string UsdtAddressTrc20 { get; set; } = string.Empty;
}

```

Файл **CoinPackageOption.cs (Quizanchos.WebApi/Options)**

Реалізація функціональної задачі – клас параметрів конфігурації пакета внутрішньоігрових монет – ідентифікатор, назва, кількість монет та ціна в USDT.

```

namespace Quizanchos.WebApi.Options;

public class CoinPackageOption
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public int Coins { get; set; }
    public decimal PriceUSDT { get; set; }
}

```

Файл **TopUpOrder.cs (Quizanchos.Domain/Entities)**

Реалізація функціональної задачі – сутність замовлення на поповнення балансу з відстеженням статусу, суми платежу, мережі та ідентифікатора блокчейн-транзакції.

```

using Quizanchos.Domain.Entities.Interfaces;

namespace Quizanchos.Domain.Entities;

public class TopUpOrder : IKeyedEntity<Guid>
{
    public Guid Id { get; set; }
    public string ApplicationUserId { get; set; } = string.Empty;
    public ApplicationUser ApplicationUser { get; set; } = null!;
    public TopUpOrderStatus Status { get; set; }
    public int CoinsToCredit { get; set; }
    public decimal AmountUSDT { get; set; }
    public string Network { get; set; } = string.Empty;
    public DateTime CreatedAtUtc { get; set; }
    public DateTime? CompletedAtUtc { get; set; }
    public string? BinanceTxId { get; set; }
}

```

Файл TopUpOrderConfiguration.cs (Quizanchos.Domain/Configurations)

Реалізація функціональної задачі – конфігурація відображення сутності TopUpOrder на таблицю PostgreSQL засобами Fluent API EF Core, включно з обмеженнями цілісності та індексом пошуку платежів.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Quizanchos.Domain.Entities;

namespace Quizanchos.Domain.Configurations;

public class TopUpOrderConfiguration : IEntityTypeConfiguration<TopUpOrder>
{
    public void Configure(EntityTypeBuilder<TopUpOrder> builder)
    {
        builder.HasKey(x => x.Id);

        builder.Property(x => x.ApplicationUserId)
            .IsRequired();

        builder.HasOne(x => x.ApplicationUser)
            .WithMany()
            .HasForeignKey(x => x.ApplicationUserId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.Property(x => x.Status)
            .IsRequired();

        builder.Property(x => x.CoinsToCredit)
            .IsRequired();

        builder.Property(x => x.AmountUSDT)
            .IsRequired()
            .HasPrecision(18, 4);

        builder.Property(x => x.Network)
            .IsRequired()
            .HasMaxLength(10);

        builder.Property(x => x.CreatedAtUtc)
            .IsRequired();

        builder.Property(x => x.CompletedAtUtc)
            .IsRequired(false);

        builder.Property(x => x.BinanceTxId)
            .IsRequired(false)
            .HasMaxLength(256);

        builder.HasIndex(x => new { x.AmountUSDT, x.Network, x.Status });

        builder.ToTable(t =>
        {
            t.HasCheckConstraint("CK_TopUpOrders_AmountUSDT_Positive", "\"AmountUSDT\" > 0");
            t.HasCheckConstraint("CK_TopUpOrders_CoinsToCredit_Positive", "\"CoinsToCredit\" > 0");
        });
    }
}
```

Файл TopUpOrderRepository.cs (Quizanchos.Domain/Repositories/Implementations)

Реалізація функціональної задачі – репозиторій доступу до даних замовлень поповнення з типовими запитами пошуку очікуваних, прострочених і завершених замовлень.

```
using Microsoft.EntityFrameworkCore;
using Quizanchos.Domain.Entities;
using Quizanchos.Domain.Repositories.Interfaces;

namespace Quizanchos.Domain.Repositories.Implementations;
```

```

public class TopUpOrderRepository : EntityRepositoryBase<Guid, TopUpOrder>, ITopUpOrderRepository
{
    public TopUpOrderRepository(QuizanchosDbContext dbContext) : base(dbContext)
    {
    }

    public async Task<TopUpOrder?> FindPendingByAmountAndNetworkAsync(decimal amount, string
network)
    {
        return await _dbSet.FirstOrDefaultAsync(x =>
            x.Status == TopUpOrderStatus.Pending &&
            x.AmountUSDT == amount &&
            x.Network == network).ConfigureAwait(false);
    }

    public async Task<List<TopUpOrder>> GetPendingByUserIdAsync(string userId)
    {
        return await _dbSet
            .Where(x => x.ApplicationUserId == userId && x.Status == TopUpOrderStatus.Pending)
            .OrderByDescending(x => x.CreatedAtUtc)
            .ToListAsync().ConfigureAwait(false);
    }

    public async Task<List<TopUpOrder>> GetAllPendingAsync()
    {
        return await _dbSet
            .Include(x => x.ApplicationUser)
            .Where(x => x.Status == TopUpOrderStatus.Pending)
            .OrderByDescending(x => x.CreatedAtUtc)
            .ToListAsync().ConfigureAwait(false);
    }

    public async Task<List<TopUpOrder>> GetExpiredCandidatesAsync(DateTime cutoffUtc)
    {
        return await _dbSet
            .Where(x => x.Status == TopUpOrderStatus.Pending && x.CreatedAtUtc < cutoffUtc)
            .ToListAsync().ConfigureAwait(false);
    }

    public async Task<bool> ExistsByBinanceTxIdAsync(string txId)
    {
        return await _dbSet.AnyAsync(x => x.BinanceTxId == txId).ConfigureAwait(false);
    }

    public async Task<List<TopUpOrder>> GetNonPendingAsync(int take)
    {
        return await _dbSet
            .Include(x => x.ApplicationUser)
            .Where(x => x.Status != TopUpOrderStatus.Pending)
            .OrderByDescending(x => x.CompletedAtUtc ?? x.CreatedAtUtc)
            .Take(take)
            .ToListAsync().ConfigureAwait(false);
    }
}

```

Файл CreateTopUpOrderRequestDto.cs (Quizanchos.WebApi/Dto/TopUp)

Реалізація функціональної задачі – об’єкт передачі даних запиту на створення замовлення поповнення.

```

namespace Quizanchos.WebApi.Dto.TopUp;

public record CreateTopUpOrderRequestDto(int PackageId, string Network);

```

Файл CreateTopUpOrderResultDto.cs (Quizanchos.WebApi/Dto/TopUp)

Реалізація функціональної задачі – об’єкт передачі даних результату створення замовлення з адресою гаманця, унікальною сумою платежу та часом завершення дії замовлення.

```

namespace Quizanchos.WebApi.Dto.TopUp;

```

```
public record CreateTopUpOrderResultDto(
    Guid OrderId,
    string WalletAddress,
    decimal AmountUSDT,
    string Network,
    DateTime ExpiresAtUtc);
```

Файл CoinPackageDto.cs (Quizanchos.WebApi/Dto/TopUp)

Реалізація функціональної задачі – об’єкт передачі даних пакета монет, доступного для придбання.

```
namespace Quizanchos.WebApi.Dto.TopUp;

public record CoinPackageDto(int Id, string Name, int Coins, decimal PriceUSDT);
```

Файл PendingTopUpOrderDto.cs (Quizanchos.WebApi/Dto/TopUp)

Реалізація функціональної задачі – об’єкт передачі даних активного (очікуваного) замовлення поповнення користувача.

```
namespace Quizanchos.WebApi.Dto.TopUp;

public record PendingTopUpOrderDto(
    Guid OrderId,
    decimal AmountUSDT,
    string Network,
    int CoinsToCredit,
    DateTime CreatedAtUtc,
    DateTime ExpiresAtUtc);
```

Файл TopUpOrderStatusDto.cs (Quizanchos.WebApi/Dto/TopUp)

Реалізація функціональної задачі – об’єкт передачі даних поточного статусу замовлення та актуального балансу монет користувача.

```
namespace Quizanchos.WebApi.Dto.TopUp;

public record TopUpOrderStatusDto(Guid OrderId, string Status, int CoinsBalance);
```

Файл AdminPendingOrderDto.cs (Quizanchos.WebApi/Dto/TopUp)

Реалізація функціональної задачі – об’єкт передачі даних очікуваного замовлення для адміністративної панелі.

```
namespace Quizanchos.WebApi.Dto.TopUp;

public record AdminPendingOrderDto(
    Guid OrderId,
    string UserId,
    string UserName,
    decimal AmountUSDT,
    string Network,
    int CoinsToCredit,
    DateTime CreatedAtUtc);
```

Файл AdminOrderHistoryDto.cs (Quizanchos.WebApi/Dto/TopUp)

Реалізація функціональної задачі – об’єкт передачі даних запису історії оброблених замовлень для адміністративної панелі.

```
namespace Quizanchos.WebApi.Dto.TopUp;

public record AdminOrderHistoryDto(
    Guid OrderId,
```



```

string UserId,
string UserName,
decimal AmountUSDT,
string Network,
int CoinsToCredit,
string Status,
DateTime CreatedAtUtc,
DateTime? CompletedAtUtc);

```

Файл TopUpService.cs (Quizanchos.WebApi/Services/Payment)

Реалізація функціональної задачі – реалізація бізнес-логіки поповнення балансу – створення замовлень з унікальною сумою, зіставлення виявлених депозитів, ручне підтвердження та анулювання прострочених замовлень у межах транзакції бази даних.

```

using System.Security.Claims;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Options;
using Quizanchos.Common.Util;
using Quizanchos.Domain;
using Quizanchos.Domain.Entities;
using Quizanchos.Domain.Repositories.Interfaces;
using Quizanchos.WebApi.Dto.TopUp;
using Quizanchos.WebApi.Options;
using Quizanchos.WebApi.Services.Users;

namespace Quizanchos.WebApi.Services.Payment;

public class TopUpService
{
    private const int OrderExpiryMinutes = 30;
    private static readonly string[] ValidNetworks = ["BEP20", "TRC20"];

    private readonly ITopUpOrderRepository _orderRepository;
    private readonly UserRetrieverService _userRetrieverService;
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly QuizanchosDbContext _dbContext;
    private readonly List<CoinPackageOption> _packages;
    private readonly BinanceApiOptions _binanceOptions;
    private readonly ILogger<TopUpService> _logger;

    public TopUpService(
        ITopUpOrderRepository orderRepository,
        UserRetrieverService userRetrieverService,
        UserManager<ApplicationUser> userManager,
        QuizanchosDbContext dbContext,
        IOptions<List<CoinPackageOption>> packages,
        IOptions<BinanceApiOptions> binanceOptions,
        ILogger<TopUpService> logger)
    {
        _orderRepository = orderRepository;
        _userRetrieverService = userRetrieverService;
        _userManager = userManager;
        _dbContext = dbContext;
        _packages = packages.Value;
        _binanceOptions = binanceOptions.Value;
        _logger = logger;
    }

    public List<CoinPackageDto> GetPackages()
    {
        return _packages.Select(p => new CoinPackageDto(p.Id, p.Name, p.Coins,
            p.PriceUSDT)).ToList();
    }

    public async Task<CreateTopUpOrderResultDto> CreateOrderAsync(ClaimsPrincipal
        claimsPrincipal, int packageId, string network)
    {
        if (!ValidNetworks.Contains(network, StringComparer.OrdinalIgnoreCase))
        {
            throw HandledExceptionFactory.Create("Invalid network. Supported: BEP20, TRC20.");
        }
    }

```

```

var package = _packages.FirstOrDefault(p => p.Id == packageId)
    ?? throw HandledExceptionFactory.Create("Invalid package.");

string userId = _userRetrieverService.GetUserId(claimsPrincipal);
string networkUpper = network.ToUpperInvariant();

decimal uniqueAmount = await GenerateUniqueAmountAsync(package.PriceUSDT,
networkUpper).ConfigureAwait(false);

string walletAddress = networkUpper switch
{
    "BEP20" => _binanceOptions.UsdtAddressBep20,
    "TRC20" => _binanceOptions.UsdtAddressTrc20,
    _ => throw HandledExceptionFactory.Create("Invalid network.");
};

var order = new TopUpOrder
{
    Id = Guid.NewGuid(),
    ApplicationUserId = userId,
    Status = TopUpOrderStatus.Pending,
    CoinsToCredit = package.Coins,
    AmountUSDT = uniqueAmount,
    Network = networkUpper,
    CreatedAtUtc = DateTime.UtcNow,
};

await _orderRepository.Create(order).ConfigureAwait(false);

return new CreateTopUpOrderResultDto(
    order.Id,
    walletAddress,
    uniqueAmount,
    networkUpper,
    order.CreatedAtUtc.AddMinutes(OrderExpiryMinutes));
}

public async Task<TopUpOrderStatusDto> GetOrderStatusAsync(ClaimsPrincipal claimsPrincipal,
Guid orderId)
{
    string userId = _userRetrieverService.GetUserId(claimsPrincipal);
    var order = await _orderRepository.GetById(orderId).ConfigureAwait(false);

    if (order.ApplicationUserId != userId)
    {
        throw HandledExceptionFactory.Create("Order not found.");
    }

    var user = await
_userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);

    return new TopUpOrderStatusDto(order.Id, order.Status.ToString(), user.Coins);
}

public async Task<List<PendingTopUpOrderDto>> GetPendingOrdersForUserAsync(ClaimsPrincipal
claimsPrincipal)
{
    string userId = _userRetrieverService.GetUserId(claimsPrincipal);
    var orders = await
_orderRepository.GetPendingByUserIdAsync(userId).ConfigureAwait(false);

    return orders.Select(o => new PendingTopUpOrderDto(
        o.Id, o.AmountUSDT, o.Network, o.CoinsToCredit,
        o.CreatedAtUtc, o.CreatedAtUtc.AddMinutes(OrderExpiryMinutes))).ToList();
}

public async Task<List<AdminPendingOrderDto>> GetAllPendingOrdersAsync()
{
    var orders = await _orderRepository.GetAllPendingAsync().ConfigureAwait(false);

    return orders.Select(o => new AdminPendingOrderDto(
        o.Id, o.ApplicationUserId, o.ApplicationUser.UserName ?? "",
        o.AmountUSDT, o.Network, o.CoinsToCredit, o.CreatedAtUtc)).ToList();
}

public async Task<List<AdminOrderHistoryDto>> GetOrderHistoryAsync(int take = 50)
{
    var orders = await _orderRepository.GetNonPendingAsync(take).ConfigureAwait(false);

    return orders.Select(o => new AdminOrderHistoryDto(

```

```

        o.Id, o.ApplicationUserId, o.ApplicationUser.UserName ?? "",
        o.AmountUSDT, o.Network, o.CoinsToCredit, o.Status.ToString(),
        o.CreatedAtUtc, o.CompletedAtUtc)).ToList();
    }

    public async Task ConfirmOrderAsync(Guid orderId, string? txId)
    {
        await using var transaction = await
        _dbContext.Database.BeginTransactionAsync().ConfigureAwait(false);

        try
        {
            var order = await _orderRepository.GetById(orderId).ConfigureAwait(false);

            if (order.Status != TopUpOrderStatus.Pending)
            {
                throw HandledExceptionFactory.Create("Order is not in pending state.");
            }

            var user = await _dbContext.ApplicationUsers
                .FirstOrDefaultAsync(x => x.Id == order.ApplicationUserId)
                .ConfigureAwait(false)
                ?? throw HandledExceptionFactory.Create("User not found.");

            user.Coins += order.CoinsToCredit;
            order.Status = TopUpOrderStatus.ManuallyConfirmed;
            order.CompletedAtUtc = DateTime.UtcNow;
            order.BinanceTxId = txId;

            await _dbContext.SaveChangesAsync().ConfigureAwait(false);
            await transaction.CommitAsync().ConfigureAwait(false);

            _logger.LogInformation("Manually confirmed top-up order {OrderId} for user {UserId},
            credited {Coins} coins",
                orderId, order.ApplicationUserId, order.CoinsToCredit);
        }
        catch
        {
            await transaction.RollbackAsync().ConfigureAwait(false);
            throw;
        }
    }

    public async Task ProcessDetectedDepositAsync(decimal amount, string network, string txId)
    {
        if (await _orderRepository.ExistsByBinanceTxIdAsync(txId).ConfigureAwait(false))
        {
            return;
        }

        var order = await _orderRepository.FindPendingByAmountAndNetworkAsync(amount,
        network).ConfigureAwait(false);
        if (order is null)
        {
            return;
        }

        await using var transaction = await
        _dbContext.Database.BeginTransactionAsync().ConfigureAwait(false);

        try
        {
            var user = await _dbContext.ApplicationUsers
                .FirstOrDefaultAsync(x => x.Id == order.ApplicationUserId)
                .ConfigureAwait(false);

            if (user is null)
            {
                _logger.LogWarning("User {UserId} not found for top-up order {OrderId}",
                order.ApplicationUserId, order.Id);
                return;
            }

            user.Coins += order.CoinsToCredit;
            order.Status = TopUpOrderStatus.Completed;
            order.CompletedAtUtc = DateTime.UtcNow;
            order.BinanceTxId = txId;

            await _dbContext.SaveChangesAsync().ConfigureAwait(false);
            await transaction.CommitAsync().ConfigureAwait(false);
        }
    }

```

```

        _logger.LogInformation("Auto-detected deposit {TxId}, credited {Coins} coins to user
{UserId}",
            txId, order.CoinsToCredit, order.ApplicationUserId);
    }
    catch (Exception ex)
    {
        await transaction.RollbackAsync().ConfigureAwait(false);
        _logger.LogError(ex, "Failed to process deposit {TxId} for order {OrderId}", txId,
order.Id);
        throw;
    }
}

public async Task ExpireStaleOrdersAsync()
{
    DateTime cutoff = DateTime.UtcNow.AddMinutes(-OrderExpiryMinutes);
    var staleOrders = await
_orderRepository.GetExpiredCandidatesAsync(cutoff).ConfigureAwait(false);

    foreach (var order in staleOrders)
    {
        order.Status = TopUpOrderStatus.Expired;
    }

    if (staleOrders.Count > 0)
    {
        await _dbContext.SaveChangesAsync().ConfigureAwait(false);
        _logger.LogInformation("Expired {Count} stale top-up orders", staleOrders.Count);
    }
}

private async Task<decimal> GenerateUniqueAmountAsync(decimal basePrice, string network)
{
    var random = new Random();

    for (int attempt = 0; attempt < 50; attempt++)
    {
        decimal suffix = random.Next(1, 100) * 0.0001m;
        decimal candidate = Math.Round(basePrice + suffix, 4);

        var existing = await _orderRepository.FindPendingByAmountAndNetworkAsync(candidate,
network).ConfigureAwait(false);
        if (existing is null)
        {
            return candidate;
        }
    }

    throw HandledExceptionFactory.Create("Unable to generate unique payment amount. Please
try again later.");
}
}

```

Файл BinanceDepositService.cs (Quizanchos.WebApi/Services/Payment)

Реалізація функціональної задачі – клієнт Binance API, що отримує історію депозитів USDT із підписуванням запитів алгоритмом HMAC-SHA256.

```

using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using Microsoft.Extensions.Options;
using Quizanchos.WebApi.Options;

namespace Quizanchos.WebApi.Services.Payment;

public class BinanceDepositService
{
    private readonly IHttpClientFactory _httpClientFactory;
    private readonly BinanceApiOptions _options;
    private readonly ILogger<BinanceDepositService> _logger;

    public BinanceDepositService(
        IHttpClientFactory httpClientFactory,
        IOptions<BinanceApiOptions> options,

```

```

        ILogger<BinanceDepositService> logger)
    {
        _httpClientFactory = httpClientFactory;
        _options = options.Value;
        _logger = logger;
    }

    public async Task<List<BinanceDeposit>> GetRecentDepositsAsync(string coin, string network,
long startTimeMs)
    {
        var client = _httpClientFactory.CreateClient("Binance");

        long timestamp = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
        string queryString =
$"coin={coin}&network={network}&startTime={startTimeMs}&status=1&timestamp={timestamp}";
        string signature = ComputeHmacSha256(queryString, _options.ApiSecret);
        string url = $"/sapi/v1/capital/deposit/hisrec?{queryString}&signature={signature}";

        using var request = new HttpRequestMessage(HttpMethod.Get, url);
        request.Headers.Add("X-MBX-APIKEY", _options.ApiKey);

        using var response = await client.SendAsync(request).ConfigureAwait(false);

        if (!response.IsSuccessStatusCode)
        {
            string body = await response.Content.ReadAsStringAsync().ConfigureAwait(false);
            _logger.LogWarning("Binance deposit history API returned {StatusCode}: {Body}",
response.StatusCode, body);
            return [];
        }

        string json = await response.Content.ReadAsStringAsync().ConfigureAwait(false);
        var deposits = JsonSerializer.Deserialize<List<BinanceDeposit>>(json, new
JsonSerializerOptions
        {
            PropertyNameCaseInsensitive = true,
            NumberHandling = JsonNumberHandling.AllowReadingFromString
        });

        return deposits ?? [];
    }

    private static string ComputeHmacSha256(string data, string secret)
    {
        byte[] keyBytes = Encoding.UTF8.GetBytes(secret);
        byte[] dataBytes = Encoding.UTF8.GetBytes(data);
        using var hmac = new HMACSHA256(keyBytes);
        byte[] hash = hmac.ComputeHash(dataBytes);
        return Convert.ToStringLower(hash);
    }
}

public class BinanceDeposit
{
    public decimal Amount { get; set; }
    public string Coin { get; set; } = string.Empty;
    public string Network { get; set; } = string.Empty;
    public int Status { get; set; }
    public string TxId { get; set; } = string.Empty;
    public long InsertTime { get; set; }
}

```

Файл DepositCheckerJob.cs (Quizanchos.WebApi/Services/Payment)

Реалізація функціональної задачі – фонове завдання Quartz.NET, що періодично опитує Binance API на предмет нових депозитів і передає їх на обробку до TopUpService.

```

using Microsoft.Extensions.Options;
using Quartz;
using Quizanchos.WebApi.Options;

namespace Quizanchos.WebApi.Services.Payment;

public class DepositCheckerJob : IJob
{

```

```

private static readonly string[] Networks = ["BEP20", "TRC20"];

private readonly BinanceDepositService _binanceService;
private readonly TopUpService _topUpService;
private readonly BinanceApiOptions _options;
private readonly ILogger<DepositCheckerJob> _logger;

public DepositCheckerJob(
    BinanceDepositService binanceService,
    TopUpService topUpService,
    IOptions<BinanceApiOptions> options,
    ILogger<DepositCheckerJob> logger)
{
    _binanceService = binanceService;
    _topUpService = topUpService;
    _options = options.Value;
    _logger = logger;
}

public async Task Execute(IJobExecutionContext context)
{
    if (string.IsNullOrEmpty(_options.ApiKey) || string.IsNullOrEmpty(_options.ApiSecret))
    {
        return;
    }

    long startTimeMs = DateTimeOffset.UtcNow.AddMinutes(-35).ToUnixTimeMilliseconds();

    foreach (string network in Networks)
    {
        try
        {
            var deposits = await _binanceService.GetRecentDepositsAsync("USDT", network,
startTimeMs).ConfigureAwait(false);

            foreach (var deposit in deposits)
            {
                try
                {
                    await _topUpService.ProcessDetectedDepositAsync(deposit.Amount, network,
deposit.TxId).ConfigureAwait(false);
                }
                catch (Exception ex)
                {
                    _logger.LogError(ex, "Failed to process deposit {TxId}", deposit.TxId);
                }
            }
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Failed to fetch {Network} deposits from Binance", network);
        }
    }
}

```

Файл OrderExpiryJob.cs (Quizanchos.WebApi/Services/Payment)

Реалізація функціональної задачі – фонове завдання Quartz.NET, що періодично переводить прострочені замовлення поповнення у статус Expired.

```

using Quartz;

namespace Quizanchos.WebApi.Services.Payment;

public class OrderExpiryJob : IJob
{
    private readonly TopUpService _topUpService;
    private readonly ILogger<OrderExpiryJob> _logger;

    public OrderExpiryJob(TopUpService topUpService, ILogger<OrderExpiryJob> logger)
    {
        _topUpService = topUpService;
        _logger = logger;
    }

    public async Task Execute(IJobExecutionContext context)

```

```

    {
        try
        {
            await _topUpService.ExpireStaleOrdersAsync().ConfigureAwait(false);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Failed to expire stale top-up orders");
        }
    }
}

```

Файл TopUpController.cs (Quizanchos.WebApi/Controllers/Payment)

Реалізація функціональної задачі – REST-контролер користувача для перегляду пакетів монет, створення замовлень поповнення та відстеження їхнього статусу.

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Quizanchos.WebApi.Constants;
using Quizanchos.WebApi.Dto.TopUp;
using Quizanchos.WebApi.Services.Payment;

namespace Quizanchos.WebApi.Controllers.Payment;

[Route("api/[controller]")]
[ApiController]
[Authorize(AppRole.User)]
public class TopUpController : ControllerBase
{
    private readonly TopUpService _topUpService;

    public TopUpController(TopUpService topUpService)
    {
        _topUpService = topUpService;
    }

    [HttpGet("packages")]
    public IActionResult GetPackages()
    {
        var packages = _topUpService.GetPackages();
        return Ok(packages);
    }

    [HttpPost("create-order")]
    public async Task<IActionResult> CreateOrder([FromBody] CreateTopUpOrderRequestDto request)
    {
        var result = await _topUpService.CreateOrderAsync(User, request.PackageId,
request.Network).ConfigureAwait(false);
        return Ok(result);
    }

    [HttpGet("order/{orderId}/status")]
    public async Task<IActionResult> GetOrderStatus(Guid orderId)
    {
        var result = await _topUpService.GetOrderStatusAsync(User,
orderId).ConfigureAwait(false);
        return Ok(result);
    }

    [HttpGet("pending")]
    public async Task<IActionResult> GetPendingOrders()
    {
        var result = await
_topUpService.GetPendingOrdersForUserAsync(User).ConfigureAwait(false);
        return Ok(result);
    }
}

```

Файл TopUpAdminController.cs (Quizanchos.WebApi/Controllers/Payment)
 Реалізація функціональної задачі – REST-контролер адміністратора для перегляду очікуваних замовлень, історії оброблених замовлень та ручного підтвердження поповнень.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Quizanchos.WebApi.Constants;
using Quizanchos.WebApi.Services.Payment;

namespace Quizanchos.WebApi.Controllers.Payment;

[Route("api/admin/topup")]
[ApiController]
[Authorize(AppRole.Admin)]
public class TopUpAdminController : ControllerBase
{
    private readonly TopUpService _topUpService;

    public TopUpAdminController(TopUpService topUpService)
    {
        _topUpService = topUpService;
    }

    [HttpGet("pending")]
    public async Task<IActionResult> GetPendingOrders()
    {
        var result = await _topUpService.GetAllPendingOrdersAsync().ConfigureAwait(false);
        return Ok(result);
    }

    [HttpGet("history")]
    public async Task<IActionResult> GetOrderHistory()
    {
        var result = await _topUpService.GetOrderHistoryAsync().ConfigureAwait(false);
        return Ok(result);
    }

    [HttpPost("{orderId}/confirm")]
    public async Task<IActionResult> ConfirmOrder(Guid orderId, [FromQuery] string? txId)
    {
        await _topUpService.ConfirmOrderAsync(orderId, txId).ConfigureAwait(false);
        return Ok(new { Message = "Order confirmed and coins credited." });
    }
}
```


Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ОНЛАЙН ПЛАТФОРМИ МІНІГОР З
ПІДТРИМКОЮ МУЛЬТИПЛЕЄРНОЇ ВЗАЄМОДІЇ ТА СИСТЕМИ
ВІРТУАЛЬНИХ АКТИВІВ (комплексна тема)**

Програма та методика тестування

КП.ІП-2116. 045440.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

_____ Максим

ГОЛОВЧЕНКО

Виконавець:

_____ Данило ПАВЛУСЕНКО

Київ – 2026

ЗМІСТ

1	ОБ’ЄКТ ВИПРОБУВАНЬ.....	3
2	МЕТА ТЕСТУВАННЯ	4
3	МЕТОДИ ТЕСТУВАННЯ.....	6
4	ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ.....	8

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є два сервіси серверного застосунку онлайн-платформи мініігор Quizanchos, розроблені автором у межах індивідуальної частини дипломного проєкту: підсистема динамічного підвантаження сторонніх мініігор-плагінів (PluginSystem) та сервіс поповнення внутрішнього балансу користувача криптовалютою USDT (Payment). Обидва сервіси входять до складу серверного модуля Quizanchos.WebApi, реалізованого мовою програмування C# 14 на платформі ASP.NET Core 10, та функціонують на серверах під керуванням операційних систем Windows і Linux.

До складу об'єкта випробування підсистеми PluginSystem входять клас динамічного сканування й завантаження збірок плагінів PluginLoader, ізольований контекст завантаження PluginLoadContext, модель завантаженого плагіна LoadedPlugin, реєстр завантажених плагінів IPluginCatalog, опис маніфесту PluginManifest, а також контракт інтеграції сторонньої мінігри IMinigameDescriptor зі складу SDK-пакета Quizanchos.Core. До складу об'єкта випробування сервісу Payment входять сервіс бізнес-логіки поповнення TopUpService, клієнт біржового застосункового інтерфейсу BinanceDepositService, фонові завдання Quartz.NET DepositCheckerJob та OrderExpiryJob, REST-контролери TopUpController і TopUpAdminController, а також сутність замовлення TopUpOrder із відповідним репозиторієм. Взаємодія сервісів зі сховищем даних здійснюється через шар Entity Framework Core 10 та СУБД PostgreSQL, а виявлення оплат – через зовнішній застосунковий інтерфейс криптовалютної біржі Binance.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є перевірка правильності реалізації функціональних та нефункціональних вимог до розроблених автором сервісів PluginSystem і Payment, сформульованих у другому розділі пояснювальної записки, з виявленням невідповідностей фактичної поведінки очікуваних. Зокрема:

- перевірка коректності динамічного виявлення та завантаження збірок сторонніх мініігор-плагінів класом PluginLoader, у тому числі реєстрації дескрипторів через інтерфейс IMinigameDescriptor у реєстрах IMinigameRegistry та IMinigameFrontendRegistry;
- перевірка стійкості процесу сканування каталогу плагінів до некоректних вхідних даних (відсутній чи порожній каталог, папка плагіна без вхідного асемблі, пошкоджений файл-маніфест plugin.json) без блокування старту серверного застосунку;
- перевірка коректності правила ізоляції збірок між хостом та ізольованим контекстом завантаження PluginLoadContext, що забезпечує рекурсивну рівність типів дескрипторів між ядром платформи й плагінами;
- перевірка дотримання інваріанти зарезервованого діапазону ідентифікаторів MinigameTypeId ≥ 1000 для сторонніх плагінів та відхилення плагінів, що цю інваріанту порушують;
- перевірка коректності створення замовлень на поповнення балансу сервісом TopUpService, у тому числі валідації мережі переказу за білим списком (BER20, TRC20), валідації обраного пакета монет та генерування унікальної суми USDT для однозначного зіставлення депозиту із замовленням;
- перевірка коректності підписування запитів до застосункового інтерфейсу біржі Binance алгоритмом HMAC-SHA256, зарахування коштів після підтвердженого депозиту й автоматичного прострочення неоплачених замовлень;
- перевірка відповідності нефункціональним вимогам щодо надійності та безпеки сервісів (атомарність нарахування коштів у межах

транзакції бази даних, ідемпотентність обробки депозитів за ідентифікатором транзакції, ізоляція сторонніх плагінів від прямого доступу до бази даних), а також знаходження дефектів, помилок і архітектурних недоліків з метою їх усунення до моменту впровадження.

3 МЕТОДИ ТЕСТУВАННЯ

Відповідно до піраміди тестування для перевірки сервісів PluginSystem і Payment застосовано автоматизоване модульне тестування серверної логіки на базі фреймворка xUnit.net, винесене у відокремлений тестовий проєкт Quizanchos.WebApi.Tests, а також серію мануальних функціональних тестів, що відтворюють роботу сервісів в умовах, наближених до реального використання платформи.

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – аналіз вихідного коду сервісів PluginSystem і Payment у складі модуля Quizanchos.WebApi з вимірюванням цикломатичної складності та метрики Холстеда без виконання програми;
- динамічне тестування – перевірка коректності роботи сервісів у процесі виконання з фіксацією помилок під час автоматизованого прогону модульних тестів та мануального відтворення сценаріїв підключення плагінів і поповнення балансу;
- функціональне тестування – перевірка відповідності фактичної поведінки сервісів сформульованим у другому розділі вимогам шляхом проходження сценаріїв підключення сторонньої мінігри, обробки некоректних плагінів та поповнення внутрішнього балансу криптовалютою USDT;
- мануальне тестування – виконання тест-кейсів вручну без автоматизації; застосовується для перевірки сценаріїв поповнення балансу у браузері та сценаріїв підключення плагінів шляхом розміщення збірок у каталозі плагінів і перезавантаження серверного застосунку;
- тестування «чорної скриньки» – перевірка REST-ендпоінтів контролерів TopUpController і TopUpAdminController виключно за вхідними і вихідними даними, без аналізу внутрішньої реалізації компонента;
- тестування «білої скриньки» – модульне тестування внутрішньої поведінки класів PluginLoader, PluginLoadContext, PluginManifest, TopUpService та логіки підписування запитів алгоритмом HMAC-SHA256 з

прямим доступом до внутрішнього стану через інстанціювання у тестах xUnit.net;

- тестування «сірої скриньки» – перевірка сканування каталогу плагінів на тимчасовій директорії та бізнес-логіки поповнення з легкою реалізацією репозиторію `ITopUpOrderRepository` в оперативній пам'яті, з частковим доступом до внутрішньої структури системи;

- регресійне тестування – повторний автоматизований запуск усього набору з 31 модульного тесту проєкту `Quizanchos.WebApi.Tests` перед внесенням змін з метою підтвердження відсутності деградації функціональності.

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Під час проведення тестування будуть використовуватись наступні допоміжні засоби:

Таблиця 4.1 – Перелік допоміжних засобів тестування

Назва засобу	Призначення в проєкті Quizanchos
xUnit.net 2.9	Автоматизоване модульне тестування сервісів PluginSystem і Payment на платформі .NET 10 за шаблоном Arrange–Act–Assert
dotnet test	Запуск набору автоматизованих модульних тестів проєкту Quizanchos.WebApi.Tests та контроль успішності їх проходження
Lizard	Статичний аналіз вихідного коду сервісів PluginSystem і Payment: цикломатична складність, метрика Холстеда, виявлення складних у супроводі ділянок
dotnet-counters	Динамічне профілювання використання ресурсів серверного процесу під навантаженням: оперативна пам'ять, пул потоків, паузи збиральника сміття
Postman, Swagger UI	Мануальне тестування REST-ендпоінтів контролерів TopUpController і TopUpAdminController з контролем структури запитів і відповідей у форматі JSON
Google Chrome (Chromium 132), DevTools	Мануальне функціональне тестування клієнтських сценаріїв поповнення внутрішнього балансу та інспекція мережевих запитів до серверного API

Продовження таблиці 4.1

.NET CLI (dotnet publish)	Складання збірок плагінів-мініігор та їх розміщення у каталозі плагінів для перевірки динамічного підвантаження підсистемою PluginSystem
GitHub Actions	Автоматизований запуск набору модульних тестів серверного веб-застосунку при злитті змін до основної гілки master репозиторію

Порядок проведення тестування є наступним:

- 1) статичний аналіз вихідного коду сервісів PluginSystem і Payment (вимірювання цикломатичної складності та метрики Холстеда інструментом Lizard, виявлення стилістичних і потенційних логічних дефектів засобами Roslyn-аналізаторів, інтегрованих у Visual Studio 2022);
- 2) автоматизоване модульне тестування у проєкті Quizanchos.WebApi.Tests із використанням фреймворка xUnit.net 2.9 за шаблоном Arrange–Act–Assert – прогін 31 тесту, згрупованого у п'ять тестових класів (PluginLoaderTests, PluginLoadContextTests, PluginManifestTests, BinanceSignatureTests та TopUpServiceTests);
- 3) перевірка стійкості сканування каталогу плагінів класом PluginLoader до некоректних вхідних даних: відсутній або порожній каталог, папка плагіна без вхідного асемблі, пошкоджений файл-маніфест plugin.json;
- 4) перевірка правила ізоляції збірок контекстом PluginLoadContext (делегування контрактних збірок SDK і збірок платформи контексту за замовчуванням та завантаження власних залежностей плагіна в його ізольованому контексті) і коректності десеріалізації маніфесту plugin.json;
- 5) перевірка бізнес-логіки сервісу TopUpService: відображення каталогу пакетів монет у DTO, валідація мережі переказу за білим списком BEP20/TRC20, відхилення невідомого пакета та генерування унікальної суми USDT;

- 6) перевірка підписування запитів до Binance Sapi алгоритмом HMAC-SHA256 (відповідність обчисленого підпису контрольному вектору, детермінованість, залежність від секретного ключа та формат результату);
- 7) мануальне функціональне тестування REST-ендпоінтів контролерів TopUpController і TopUpAdminController за допомогою застосунку Postman та інтерфейсу Swagger UI;
- 8) мануальне функціональне тестування клієнтських сценаріїв поповнення балансу у браузері Google Chrome (Chromium 132): створення замовлення через мережу BEP20, зарахування коштів після підтвердженого депозиту, забезпечення унікальності суми, прострочення замовлення та відхилення запитів з невідомим пакетом чи неприпустимою мережею переказу;
- 9) тестування динамічного підвантаження сторонніх мініігор шляхом розміщення підготовлених збірок у кореневому каталозі плагінів Plugins:Root, перезавантаження серверного застосунку та контролю результату в журналі застосунку й каталозі мініігор, у тому числі відхилення плагіна із зарезервованим ідентифікатором MinigameTypeId < 1000;
- 10) відтворення контрольного прикладу повного життєвого циклу інтеграції сторонньої мінігри – від початкового стану каталогу з N мініграми до кінцевого стану з N+1 мінігрою без зміни та перекомпіляції коду ядра платформи;
- 11) регресійне тестування шляхом повторного автоматизованого запуску всього набору модульних тестів проєкту Quizanchos.WebApi.Tests у конвеєрі GitHub Actions перед кожним релізом нової версії з метою підтвердження відсутності деградації функціональності.

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ОНЛАЙН ПЛАТФОРМИ МІНІГОР З
ПІДТРИМКОЮ МУЛЬТИПЛЕЄРНОЇ ВЗАЄМОДІЇ ТА СИСТЕМИ
ВІРТУАЛЬНИХ АКТИВІВ (комплексна тема)**

Графічний матеріал

КПІ.ПІ-2116.045440.06.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

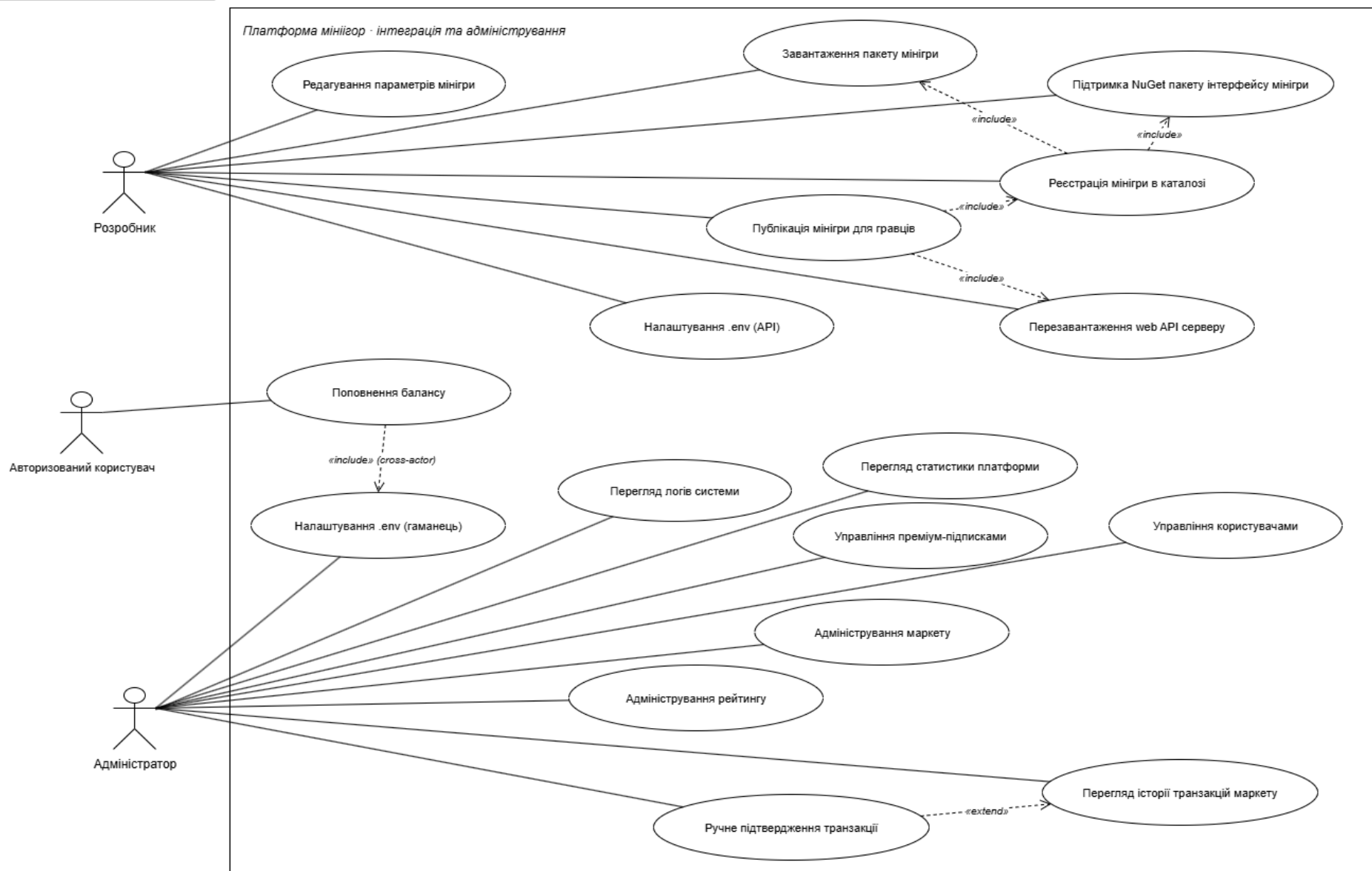
_____ Максим

ГОЛОВЧЕНКО

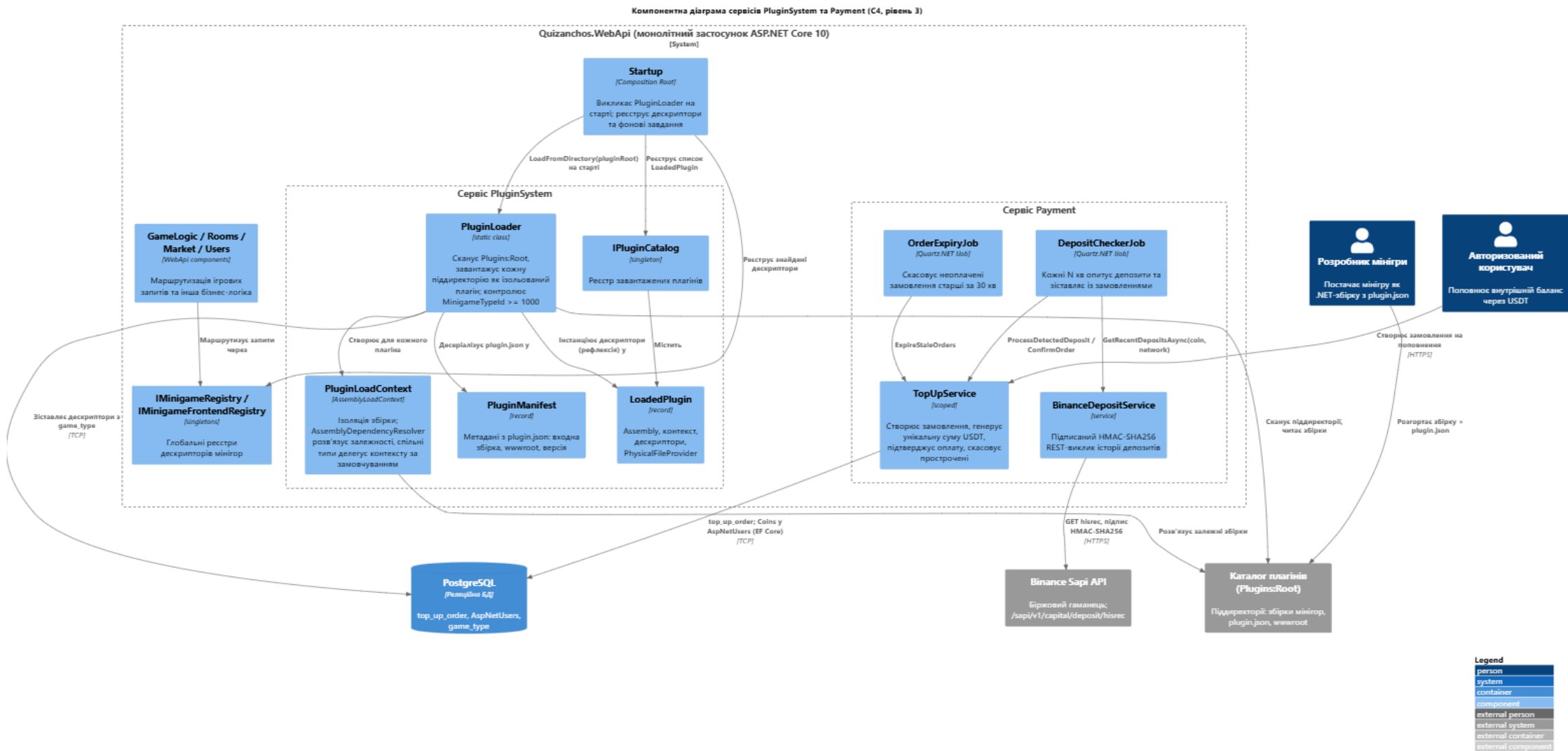
Виконавець:

_____ Данило Павлусенко

Київ – 2026

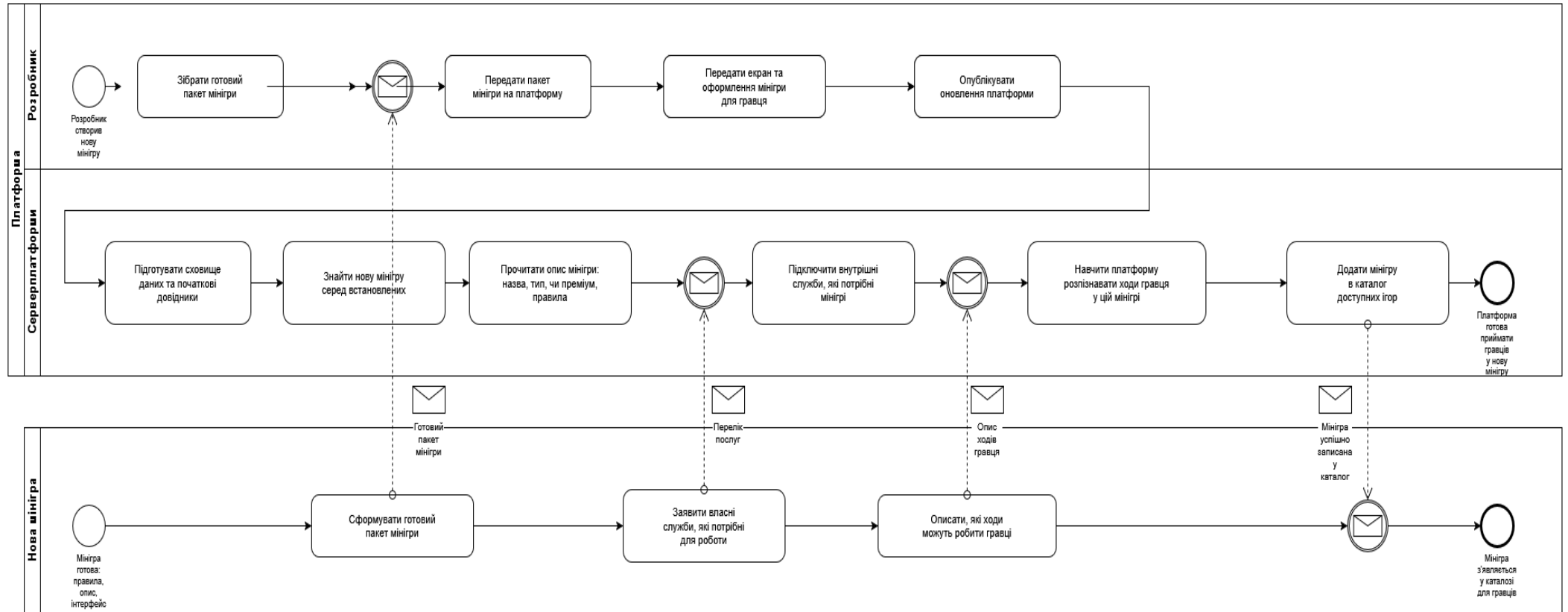


					КПІ.ІП-2116.045440.06.99.CCB					
					Схема структурна варіантів використання	Літера		Маса	Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата						
Розробив		Павлусенко Д.О.								
Перевірів		Павлов О.А.								
Т. контр.										
					Програмне забезпечення онлайн платформи мініігор з підтримкою мультиплеерної взаємодії та системи віртуальних активів(комплексна тема)	Аркуш		Аркушів		
Н. контр.		Головченко М.М.				КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-21				
Затвердив		Жаріков Е.В.								



					КП.ІП-2116.045440.06.99.CCM			
					Компонентна діаграма сервісів PluginSystem та Payment (C4, рівень 3)	Літера	Маса	Масштаб
Зм.	Арк.	№ документа	Підпис	Дата				
Розробив		Павлусенко Д.С.						
Перевірів		Павлов О.А.						
Т. контр.					Програмне забезпечення онлайн платформи мінігор з підтримкою мультиплеерної взаємодії та системи віртуальних активів(комплексна тема)	Аркуш		Аркушів
Н. контр.		Головченко М.М.				КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-21		
Затвердив		Жаріков Е.В.						

Схема бізнес-процесу розгортання нової мінігри



Демонстраційний плакат до дипломного проекту
Програмне забезпечення онлайн платформи мініігор з підтримкою мультиплеєрної взаємодії та системи віртуальних активів(комплексна тема)

Виконав студент гр. ІП-21 Павлусенко Д.О.

Керівник Павлов О.А.